AUTOMATED CRIMINAL DETECTION USING CNN-BASED FACIAL RECOGNITION SYSTEM

A project Report submitted in the partial fulfillment of the requirements

for the award of

degree in

BACHELOR OF TECHNOLOGY
(COMPUTER SCIENCE AND ENGINEERING)

Submitted By

321506402132	**ITIKARLAPALLI DEEPAK**
321506402111	**GILAKALA HARSHAVARDHAN**
321506402115	**GORREPOTU JASWANTH**
321506402119	**GRANDHI JAYANTH**

Under the guidance of

Mr. V. Nagaraju

Assistant Professor

Department of Computer Science and Systems Engineering

Andhra University College of Engineering (A)

Andhra University, Visakhapatnam

April-2025

AUTOMATED CRIMINAL DETECTION USING CNN-BASED FACIAL RECOGNITION SYSTEM

A project Report submitted in the partial fulfillment of the requirements

for the award of

degree in

BACHELOR OF TECHNOLOGY
(COMPUTER SCIENCE AND ENGINEERING)

Submitted By

321506402132	**ITIKARLAPALLI DEEPAK**
321506402111	**GILAKALA HARSHAVARDHAN**
321506402115	**GORREPOTU JASWANTH**
321506402119	**GRANDHI JAYANTH**

Under the guidance of

Mr. V. Nagaraju

Assistant Professor

Department of Computer Science and Systems Engineering

Andhra University College of Engineering (A)

Andhra University, Visakhapatnam

April-2025

Department of Computer Science and Systems Engineering

Andhra University College of Engineering (A)

Andhra University, Visakhapatnam

CERTIFICATE

This is to certify that this Project report entitled "**AUTOMATED CRIMINAL DETECTION USING CNN-BASED FACIAL RECOGNITION SYSTEM**", is the Bonafide work carried out by **ITIKARLAPALLI DEEPAK** (321506402132), **GILAKALA HARSHA VARDHAN** (321506402111) **GORREPOTU JASWANTH** (321506402115), **GRANDHI JAYANTH** (321506402119) , in partial fulfillment of the requirements for the award of degree in Bachelor of Technology (Computer Science and Engineering) in the Department of Computer Science & Systems Engineering, Andhra University College of Engineering (A), Andhra University, Visakhapatnam, during December 2024 - April 2025.

Asst. Prof. V. Nagaraju
Project Guide

Prof. Kasukurthi Venkata Rao
Head of the Department

DECLARATION

We hereby declare that the project report entitled **"AUTOMATED CRIMINAL DETECTION USING CNN-BASED FACIAL RECOGNITION SYSTEM** is an original work done at Department of Computer Science and Systems Engineering, Andhra University College of Engineering (A), Andhra University, Visakhapatnam, submitted in partial fulfillment of the requirements for the award of degree in Bachelor of Technology (Computer Science and Engineering) during December 2024 – April 2025.

Registration Number	Student Name	Signature
321506402132	**ITIKARLAPALLI DEEPAK**	
321506402111	**GILAKALA HARSHA VARDHAN**	
321506402115	**GORREPOTU JASWANTH**	
321506402119	**GRANDHI JAYANTH**	

Place: Visakhapatnam

Date:

ACKNOWLEDGEMENT

We would like to show my greatest appreciation to **Asst. Prof. V. Nagaraju** for his tremendous support and help. Without his encouragement and guidance this project would not have materialized.

We would like to thank **Prof. Kasukurthi Venkata Rao**, Head of the department of Computer Science and Engineering, Andhra University College of Engineering, for his encouragement and valuable guidelines in bringing shape to the dissertation.

We are sincerely thankful to **Prof. D. LALITHA BHASKARI**, Chairman, Board of studies, Andhra University College of Engineering, for her valuable support.

We wish to express thanks to **Prof. G. SASIBUSHANA RAO**, Principal, Andhra University College of Engineering, for helping us in completing the project work on time.

We would like to thank the teaching staff and non-teaching staff members of the department of Information Technology and Computer Applications, Andhra University College of Engineering, Visakhapatnam, for their constant support in successful completion of my study.

ITIKARLAPALLI DEEPAK	**321506402132**
GILAKALA HARSHA VARDHAN	**321506402111**
GORREPOTU JASWANTH	**321506402115**
GRANDHI JAYANTH	**321506402119**

ABSTRACT

Face is the principal distinguishing attribute of the human body in recognizing an individual. Face recognition is vital for the law enforcement authorities in primary detection and identification of criminals. However, it becomes very difficult and time-consuming to manually identify every individual in highly populous cities and public places. In this paper, we propose an automatic criminal identification technique for face recognition of the criminal's using history of their criminal activities. The proposed system will detect the detailed features of human face and recognize the suspect entering a public location. For this purpose, we have built a facial recognition system to apply in Convolutional Neural Networks (CNN) for criminal identification using deep learning models, specifically inceptionv3.

These models, renowned for their robust feature extraction capabilities, are employed to analyze and classify facial images to aid in the identification of suspects. The study involves training and validating these CNN architectures on a dataset of criminal portraits, followed by performance evaluation in terms of accuracy, precision, and recall. Our experiments demonstrate that inceptionv3, with its deeper architecture, offers improved performance over inceptionv3 in distinguishing between individuals, achieving higher accuracy rates in facial recognition tasks. This research highlights the potential of integrating sophisticated CNN models into criminal identification systems, providing a valuable tool for law enforcement agencies to enhance their investigative processes. The findings suggest that leveraging these advanced models can lead to more reliable and efficient identification solutions, contributing to the broader field of automated security and surveillance. Preliminary findings prove the feasibility of implementing this system in effective detection and identification of individuals.

Keywords: Face recognition, criminal identification, convolutional neural networks (CNN), deep learning, InceptionV3, feature extraction, computer vision, image processing

TABLE OF CONTENTS

CONTENT **PAGE NUMBER**

LIST OF FIGURES

CHAPTER-1

INTRODUCTION

1. Introduction

Criminal detection and identification remain critical challenges for law enforcement agencies, especially in densely populated urban areas and high-traffic public spaces. The task of manually identifying suspects from vast crowds is a daunting and inefficient process, often involving significant human resources and time.

1.1 Purpose

In law enforcement, face recognition has become an essential tool for detecting and identifying criminals, aiding authorities in the fight against crime. However, manual identification of suspects, particularly in highly populated cities and public spaces, remains a challenging and resource-intensive task.

1.2 Scope

This system is designed to be deployed in public areas with high surveillance coverage, such as airports, metro stations, shopping malls, and other crowded spaces. It will integrate with existing security cameras and cross-reference captured images with a criminal database to identify potential suspects. The system's applications extend to law enforcement agencies, security firms, and government institutions, ensuring efficient criminal detection and reducing investigation time.

1.3 Motivation

The increasing crime rates and the exponential growth of surveillance footage necessitate a more efficient and automated approach to criminal identification. Traditional methods of manual suspect identification are time-consuming, error-prone, and inefficient in large-scale settings. The demand for real-time, accurate, and scalable solutions has driven the need for advanced AI-powered facial recognition systems. By automating this process, law enforcement agencies can enhance public safety and respond to threats more effectively.

1.4 Fundamental Concepts

The Criminal Identification System is built upon key concepts from machine learning, computer vision, and facial recognition technologies. These concepts form the foundation of the system, enabling it to detect, process, and identify faces accurately.

1.4.1 Machine Learning and Artificial Intelligence

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that allows systems to learn patterns from data without being explicitly programmed. The Criminal Identification System employs supervised learning, where a deep learning model is trained using labeled images of criminals. By learning from these images, the model can classify new faces accurately.

1.4.2 Computer Vision

Computer Vision enables computers to interpret and analyze visual data. In this system, OpenCV (Open-Source Computer Vision Library) is used to process images and detect facial features. It allows the system to extract relevant details from images or live video feeds, ensuring accurate face detection and matching.

1.4.3 Face Detection and Recognition

Face detection is the process of identifying human faces within an image. The system uses the Haar Cascade Classifier to detect faces, which are then processed and fed into the InceptionV3 deep learning model for recognition. Face recognition involves comparing a detected face against stored criminal images and identifying matches based on extracted facial features.

1.4.4 Deep Learning and Convolutional Neural Networks (CNNs)

Deep learning is a branch of machine learning that mimics the way the human brain processes data. The InceptionV3 model, a type of Convolutional Neural Network (CNN), is used for feature extraction. CNNs are highly effective for image processing tasks, as they analyze spatial hierarchies and recognize facial patterns at different levels.

1.5 Proposed System Fundamentals

The proposed model for criminal detection uses **Convolutional Neural Networks (CNNs)**, specifically the **InceptionV3** architecture, to recognize faces in images or videos, and then cross-reference these faces with a criminal database.

1.6 Proposed Algorithms

1.6.1 Convolutional Neural Networks (CNN) – InceptionV3

The Criminal Identification System employs InceptionV3, a deep learning model designed for image classification and feature extraction. This Convolutional Neural Network (CNN) is pre-trained on ImageNet and is fine-tuned for criminal face recognition. The model processes images through multiple convolutional layers to extract distinctive facial features such as contours, eye positions, and skin texture. The extracted features are passed through fully connected layers, followed by a SoftMax activation function, which assigns a probability score to each possible match. This allows the system to effectively classify individuals based on facial data and identify them within the criminal database.

1.6.2 Image Preprocessing with OpenCV

Before feeding images into the model, the system preprocesses them using OpenCV to enhance facial features and improve recognition accuracy. The Haar Cascade Classifier is used to detect faces in uploaded images or real-time camera feeds. Once detected, the face is cropped and resized to match the model's input dimensions. Further enhancements, such as CLAHE (Contrast Limited Adaptive Histogram Equalization), are applied to improve contrast in low-light conditions, making facial details more prominent. The system also applies padding and resizing techniques to standardize image dimensions, ensuring consistency in recognition.

1.6.3 Image Augmentation using ImageDataGenerator

To make the model robust to variations in lighting, angles, and expressions, image augmentation is applied using ImageDataGenerator. Augmentation techniques such as rotation, width and height shifts, shear transformations, zooming, and horizontal flipping create variations

of the same image, helping the model generalize better. This ensures that the system can recognize faces under different conditions, such as varying facial expressions, head tilts, or changes in brightness. By exposing the model to diverse variations during training, the system achieves improved accuracy in real-world scenarios.

1.6.4 Classification using Softmax Regression

After feature extraction, the system uses SoftMax regression to classify the detected face. The SoftMax function converts the model's raw output into a probability distribution across different classes, identifying the most likely match. If a match is found with a high probability, the system displays the criminal's details; otherwise, it flags the individual as unknown. This classification process allows the system to distinguish between multiple individuals and assign each face to its correct identity within the criminal database.

1.6.5 Optimization using RMSprop

To enhance training efficiency and stability, the model is optimized using RMSprop (Root Mean Square Propagation). RMSprop dynamically adjusts the learning rate for each parameter, preventing large fluctuations and ensuring faster convergence. This optimization technique helps the model learn effectively from the available dataset, leading to more accurate predictions. The combination of transfer learning with InceptionV3 and RMSprop optimization ensures that the Criminal Identification System maintains high recognition accuracy.

1.6.6 Model Training Enhancements

During training, the system incorporates Early Stopping and Model Checkpointing techniques to improve performance. Early Stopping monitors validation accuracy and stops training if no improvement is observed over multiple epochs, preventing overfitting. Model Checkpointing saves the best-performing model during training, ensuring that the most optimized version is used for facial recognition. These techniques help in achieving a stable and reliable machine learning model for real-world deployment.

1.6.7 Criminal Identification Process

When an image or video is uploaded for identification, the system detects faces using OpenCV, preprocesses them, and passes them through the trained InceptionV3 model. The extracted facial features are compared against stored criminal records, and the system returns a match probability score for each detected individual. If a match is found, the system retrieves the corresponding criminal details; otherwise, the face is stored for further analysis. This automated identification process ensures quick and accurate recognition, aiding law enforcement in identifying criminals efficiently.

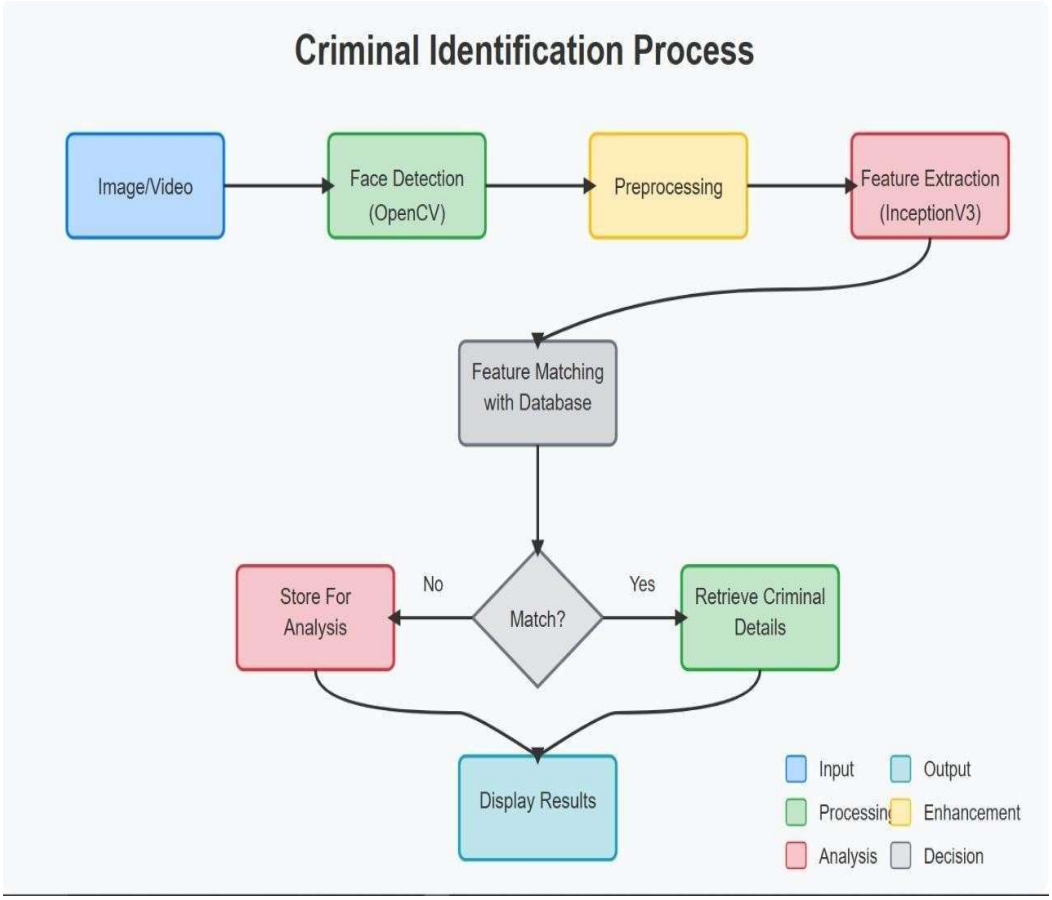


Fig 1.6.7: Criminal Identification Process

CHAPTER-2
LITERATURE SURVEY

Various approaches have been implemented by researchers to apply deep learning and computer vision techniques in addressing security and criminal identification challenges, with a particular focus on facial recognition and suspect categorization. These methods vary across research teams, employing different neural network architectures and models to enhance accuracy and efficiency in law enforcement applications. Our research specifically examines the effectiveness of Convolutional Neural Networks (CNN) for facial recognition, with emphasis on comparing the performance of InceptionV3 models in criminal identification systems.

O. M. Parkhi, A. Vedaldi, and A. Zisserman [1] proposed a new method called Deep Face Recognition to improve facial recognition accuracy. The system starts with data pre- processing, where facial images are processed and using deep convolutional neural networks (CNNs) to identify distinctive facial features. Multiple datasets are used individually and in various combinations with different deep learning architectures. The system employs CNN-based classification models. The performance of each model is evaluated using metrics like accuracy, precision, and classification error.

Yan Ke and R. Sukthankar [2] proposed a new method called PCA-SIFT to improve the distinctiveness and robustness of local image descriptors. The system starts with feature extraction, where Scale-Invariant Feature Transform (SIFT) descriptors are computed from images. Dimensionality reduction is performed using Principal Component Analysis (PCA) to enhance descriptor efficiency while retaining important information. Local image descriptors are used individually and in various combinations with different computer vision techniques. The system employs PCA-based feature transformation models. The performance of each model is evaluated using metrics like matching accuracy, robustness, and computational efficiency.

R. Chellappa, C. L. Wilson, and S. Sirohey [3] conducted a comprehensive study on human and machine recognition of faces to analyze various approaches for facial recognition. The survey begins with a review of psychological and computational aspects of face recognition, where human perception and machine-based techniques are compared. Feature extraction and classification methods are examined using statistical, structural, and neural

network-based approaches to identify key advancements in the field. Various datasets and algorithms are analyzed individually and in different combinations with pattern recognition techniques. The study evaluates system performance using metrics like recognition accuracy, complexity. robustness, and computational.

F. Schroff, D. Kalenichenko, and J. Philbin [4] proposed a new method called FaceNet to improve face recognition and clustering. The system starts with deep learning-based feature extraction, where facial images are mapped to a compact Euclidean space using a deep convolutional neural network (CNN). Feature representation is optimized using triplet loss to ensure that faces of the same identity are closer while different identities remain far apart. Face embeddings are used individually and in various combinations with different classification and clustering techniques. The system employs a CNN-based architecture trained on large-scale datasets. The performance of each model is evaluated using metrics like verification accuracy, clustering efficiency, and computational cost.

V. Bruce, Z. Henderson, and A. M. Burton [5] conducted a study on matching identities of familiar and unfamiliar faces in CCTV images to analyze the accuracy of human recognition under real-world conditions. The study starts with analyzing human performance in face matching tasks, where participants attempt to identify individuals from CCTV footage. Recognition performance is evaluated using factors like familiarity, image quality, and viewing conditions to understand the challenges in face identification. CCTV images of varying quality are used individually and in various combinations with different experimental conditions. The study assesses human accuracy using metrics like matching success rate, error rate, and response time.

CHAPTER-3
SYSTEM ANALYSIS

3.1 Existing System

The existing system for criminal identification primarily relies on **manual methods** for detecting and identifying criminals. Law enforcement agencies typically depend on **manual records**, **human judgment**, and **traditional databases** to track individuals suspected of criminal activities.

This process involves comparing facial features, criminal records, and other identifying information, often requiring **human officers** to review surveillance footage or images and manually match them with known suspects.

This method is **time-consuming** and often prone to errors, as it heavily relies on the **experience** and **attention to detail** of the individuals performing the identification. Moreover, in highly populated areas or busy public spaces, the ability to manually identify criminals in real-time is limited.

The growing volume of surveillance data further complicates the process, making it difficult for law enforcement to identify criminals efficiently or respond promptly to threats.

The **manual system** is not only inefficient but also lacks the scalability required for modern-day law enforcement, where quick identification and response are crucial to public safety.

In addition, the manual system offers **limited mobility and field access** options for officers on patrol, who often cannot quickly verify identities or access comprehensive criminal histories when encountering suspicious individuals outside the station environment. This disconnect between field operations and centralized records systems creates dangerous gaps in situational awareness.

3.2 Proposed System:

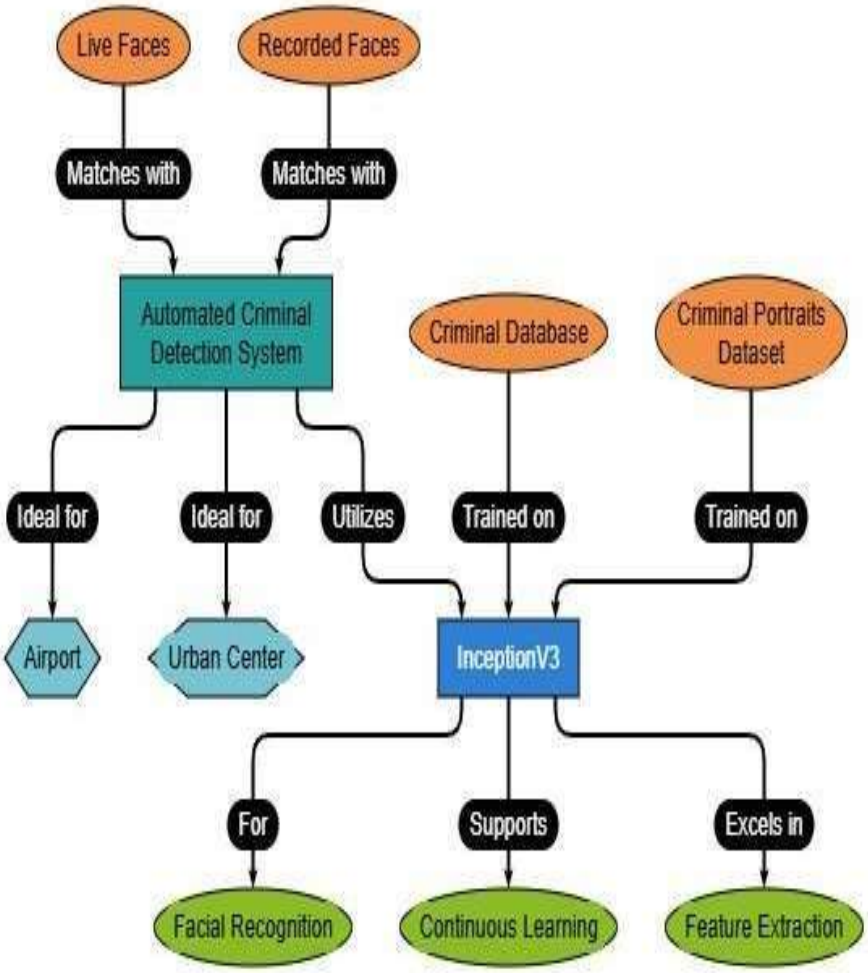


Fig 3.2.1: Proposed System

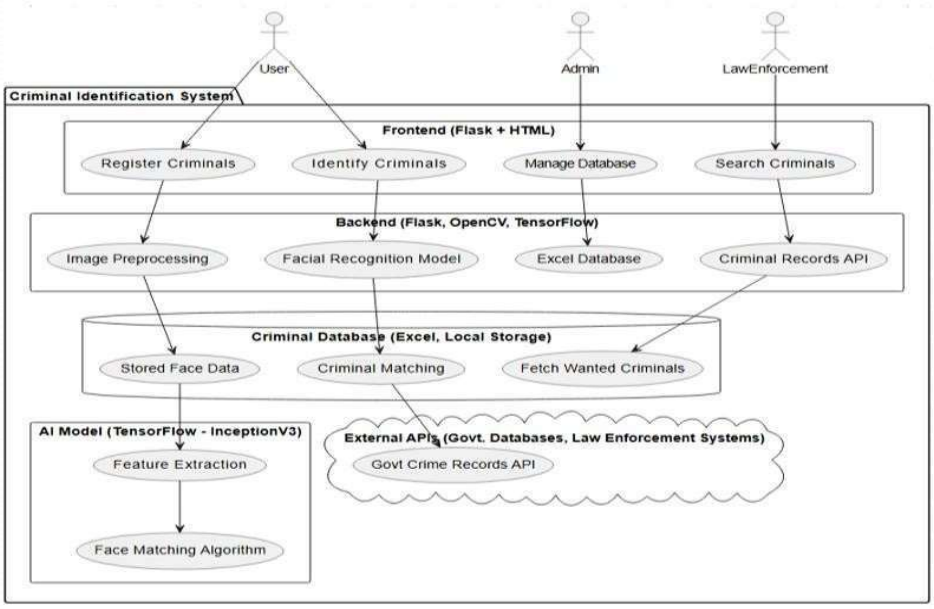


Fig 3.2.2:Block Diagram of proposed methodology

Before recognizing a face, the system first needs to detect the presence of faces in images or video frames. Once the face is detected, the system uses the **InceptionV3** model to recognize the individual.

InceptionV3, is a deep CNN that uses **inception modules** to process images at different scales, which helps capture more complex features and improve classification accuracy.

The identified face is matched against a criminal database stored in an **Excel file** or a relational database. This database contains **facial embeddings** and **criminal records** associated with each individual.

The system supports real-time face recognition, where facial images from live video feeds (e.g., from surveillance cameras) are captured frame by frame. These frames are processed in real-time, and the system compares the detected faces with the criminal database.

If a match is found, an alert is generated and displayed on the user interface, notifying law enforcement personnel of the potential suspect.

CHAPTER-4

SYSTEM REQUIREMENTS SPECIFICATIONS

4.1 Software Requirements

- **Operating System :** Windows 10/11, Linux (Ubuntu), or macOS
- **Programming Languages :** Python (for backend and ML model)
- **Database Management System**: MongoDB / MySQL / PostgreSQL (for storing user and facial data)
- **Version Control:** Git & GitHub
- **Frameworks & Libraries:**
  - OpenCV (for image processing)
  - TensorFlow/Keras or PyTorch (for facial recognition model)
  - Flask/Django (for web-based application)
  - NumPy & Pandas (for data handling)
- **API & Integration Tools:**
  - AWS Rekognition / Azure Face API (optional for cloud-based processing)
  - Development Tools & IDEs:
  - VS Code / PyCharm / Jupyter Notebook
  - Postman (for API testing)

4.2 Hardware Requirements

- **Processor:** Intel i5/i7 (10th Gen or later) / AMD Ryzen 5/7 (for smooth execution)
- **RAM:** Minimum 8GB (16GB recommended for ML model training)
- **Storage:** Minimum 256GB SSD (512GB+ recommended for faster processing)
- **GPU:** NVIDIA RTX 3060+ / Tesla T4 (if training a deep learning model locally)
- **Camera:** High-resolution Webcam / IP Camera (for real-time image capture)
- **Network:** Stable internet connection (for cloud-based facial recognition)

4.3 Pre requisites

The modules we should import are:

Python:

Python is the primary programming language for developing the entire system. It is chosen due to its extensive support for machine learning libraries and ease of integration with web technologies.

TensorFlow/Keras:

TensorFlow and Keras are used for implementing the InceptionV3 model, fine-tuning it with the criminal dataset, and deploying it for inference. TensorFlow provides powerful tools for managing deep learning models, while Keras simplifies the creation and training of models.

OpenCV:

OpenCV is used for image processing, such as detecting faces in video frames and performing basic image manipulations like resizing and cropping.

Flask:

Flask is used to create a web-based user interface for the criminal detection system. It enables law enforcement personnel to interact with the system, view live feeds, search the criminal database, and receive alerts in real-time.

Excel / SQL Database:

An Excel file or a SQL database is used for storing criminal records, face embeddings, and criminal history. SQL queries allow for efficient matching of detected faces with known criminals in the database.

Scikit-learn:

Scikit-learn is used for additional data processing and evaluation tasks, such as measuring the model's performance using metrics like accuracy, precision, recall, and F1-score.

CHAPTER-5
SYSTEM DESIGN

5.1 System Model

- Method : Deep Learning-based Facial Recognition
- Type : CNN + InceptionV3
- Dataset Used: Custom Criminal Face Dataset
- Pre-Trained Model Used: InceptionV3
- Model: Criminal Identification System

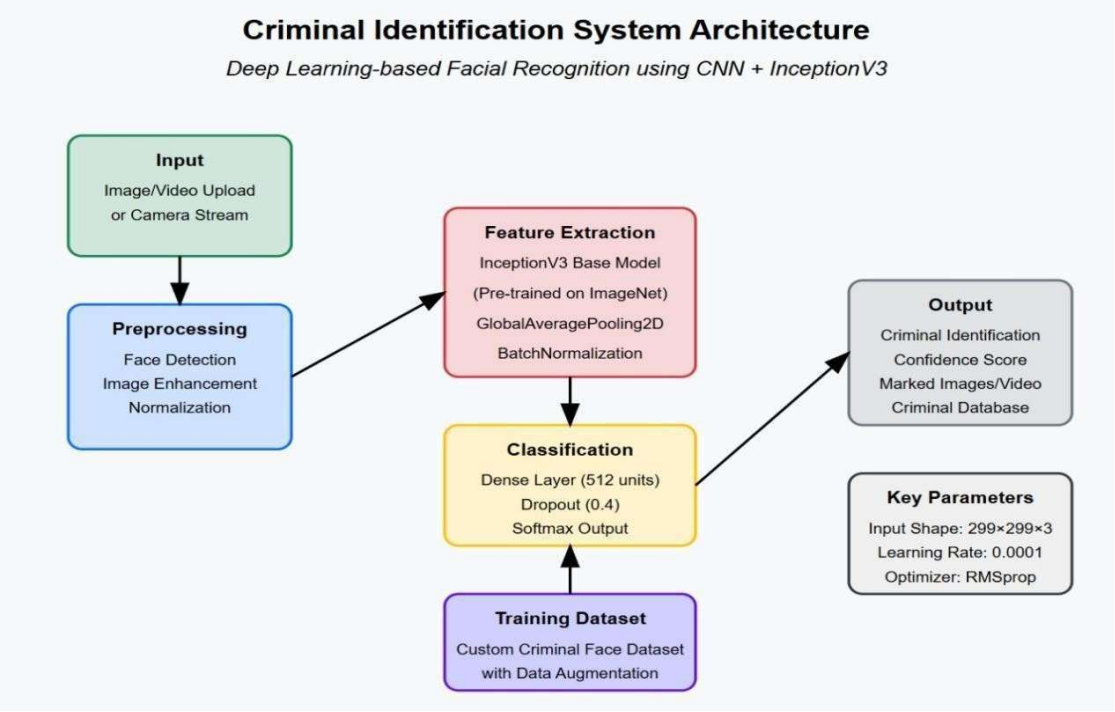


Fig 5.1.1 : System Architecture

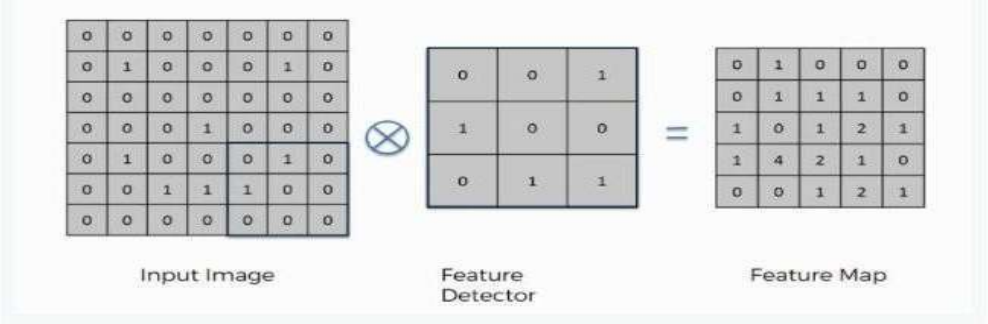
5.2 SYSTEM ARCHITECTURE

5.2.1 Convolutional Neural Network(CNN):

CNN is a Deep Learning algorithm which takes in an input image and assigns importance (learnable weights and biases) to various aspects/objects in the image, which helps it differentiate one image from the other.

The following are steps involved:

Step 1a : Convolution



It consists of input image and feature detector of 3*3, 7*7, 9*9 but considers only 3*3.Count the cells from top left 3*3 till right and top to bottom that are matched and added to first row and first col of feature map and process is repeated till all the rows are filled.

Fig 5.2.1a : Convolution

Step 1b : ReLu Layer

The Rectified Linear Unit Or ReLu is not a separate component of Convolutional Networks process but most commonly used activation function.

The main purpose of applying the rectifier function is to increase non linearity of images. The function returns 0 if it is negative else positive returns the value back. It is written as,

f(x) = max(0,x)

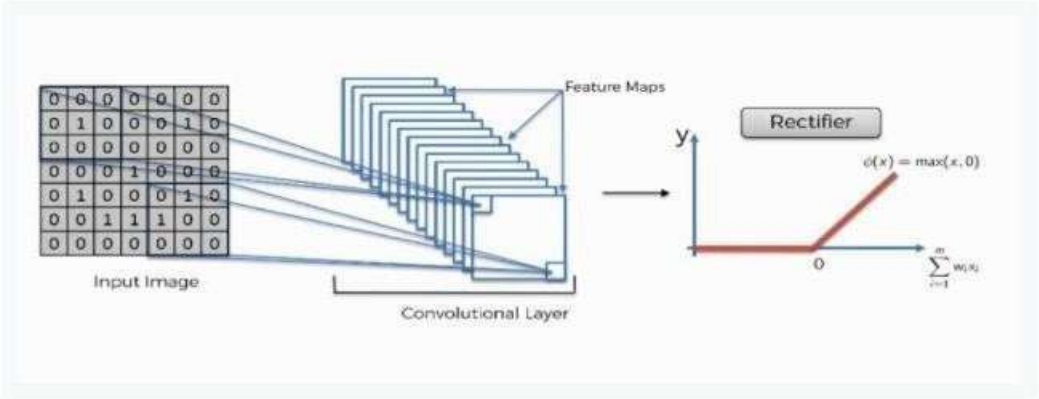


Fig 5.2.1b : ReLu Layer

Step 2: Pooling

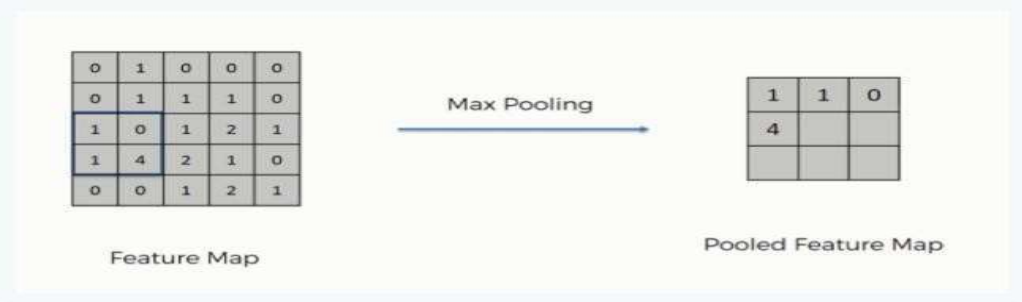


Fig 5.2.1.2 : Pooling

The process of filling in a pooled feature map differs from the one we used to come up with the regular feature map. This time you'll place a 2x2 box at the top-left corner, and move along the row.

For every 4 cells your box stands on, you'll find the maximum numerical value and insert it into the pooled feature map. For above Eg, the box currently contains a group of cells where the maximum value is 4

Step 3: Flattening

Before inserting into artificial neural network it must be flattened.

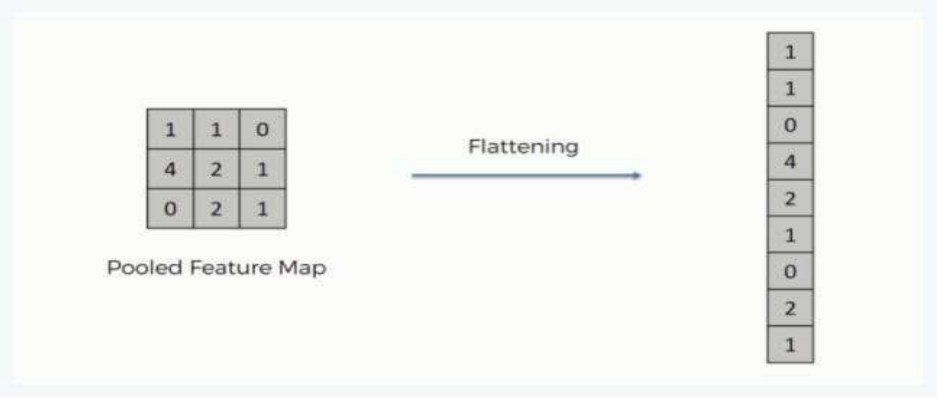


Fig 5.2.1.3 : Flattening

Step 4: Full Connection

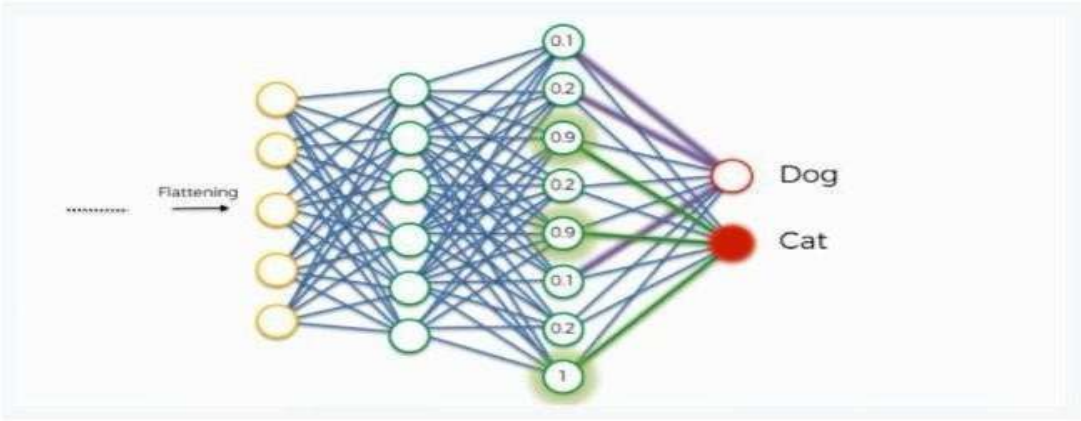


Fig 5.2.1.4 : Full Connection

This full connection works as follows:

- The neuron in the fully-connected layer detects a certain feature; say, a nose.
- It preserves its value.
- It communicates this value to both the "dog" and the "cat" classes.
- Both classes check out the feature and decide whether it's relevant to them.

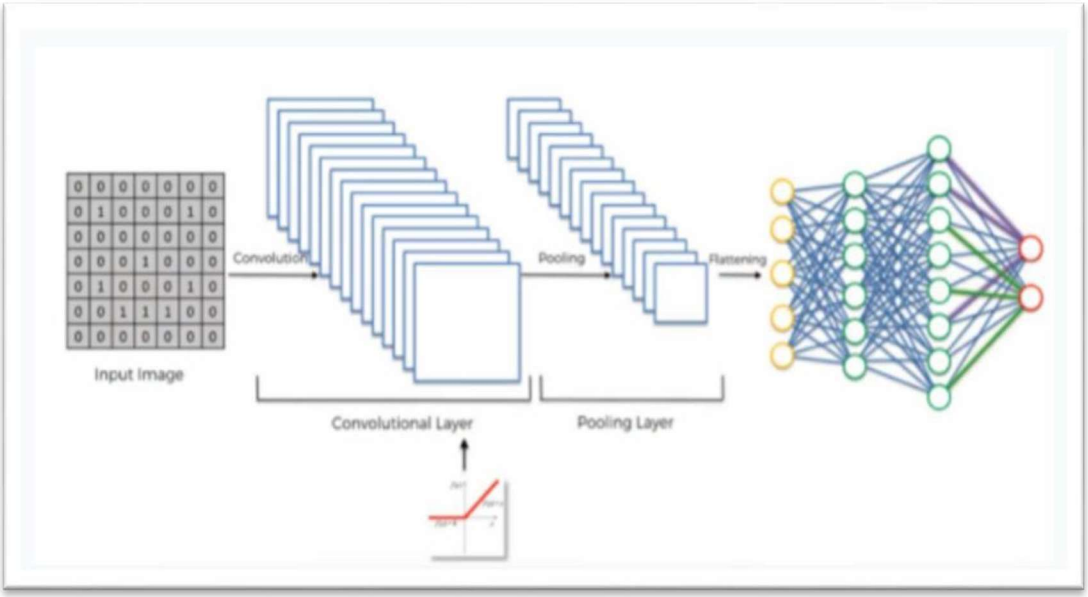


Fig 5.2.1 : CNN Architecture

5.3 UML Diagrams

UML, which stands for Unified Modeling Language, is a way to visually represent the architecture, design, and implementation of complex software systems. When you're writing code, there are thousands of lines in an application, and it's difficult to keep track of the relationships and hierarchies within a software system. UML diagrams divide that software system into components and subcomponents.

5.3.1 Use Case Diagram

A use case diagram is a visual representation of a set of use cases and actors that may be used to define the functionality and behaviour of a whole application system. The needs of a system, including internal and external factors, are gathered via use case diagrams. The majority of these criteria are design-related. As a result, when a system's functionalities are analysed. Use cases are prepared and actors are identified.

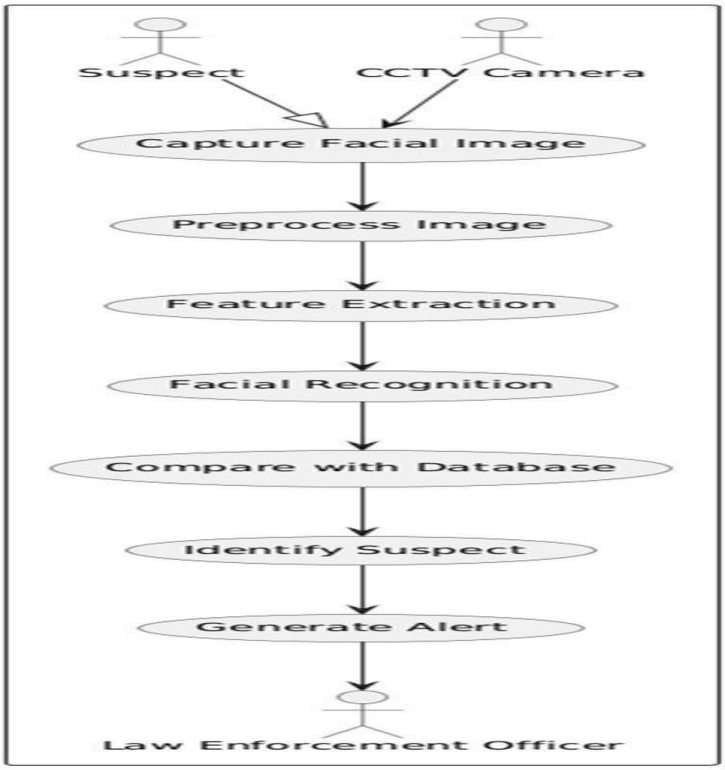


Fig 5.3.1 : Use Case Diagram

5.3.2 Activity Diagram

Activity diagrams visualize the steps performed in a use case—the activities can be sequential, branched, or concurrent. This type of UML diagram is used to show the dynamic behavior of a system, but it can also be useful in business process modeling.

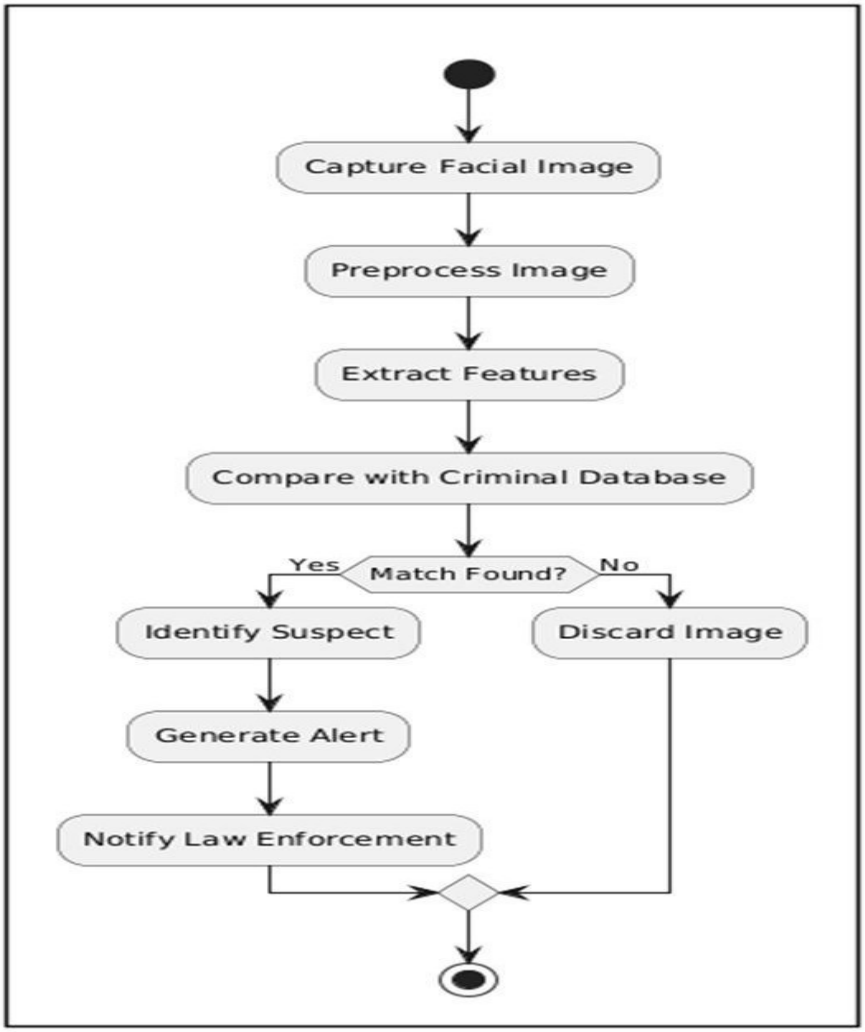


Fig 5.3.2 : Activity Diagram

5.3.3 Sequence Diagram

A sequence diagram is a type of UML (Unified Modeling Language) diagram that visually represents how objects in a system interact with each other over time. It is primarily used in software engineering to depict the flow of messages or events between different system components, helping in understanding use case scenarios, system behaviors, and interactions.

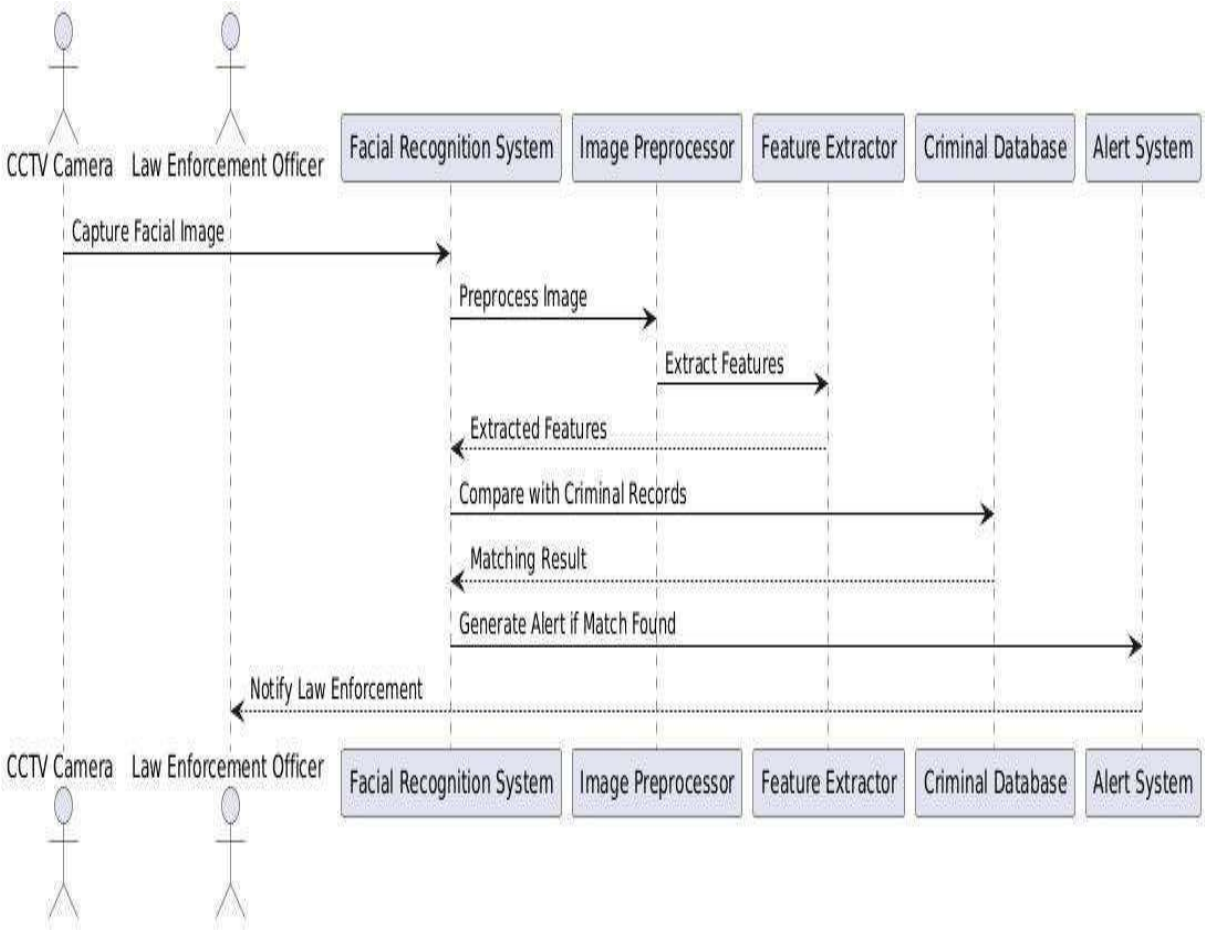


Fig5.3.3 : Sequence Diagram

5.3.4 Flow Chart Diagram

A flowchart is a diagram that depicts the steps in a process. It began as a tool for expressing algorithms and programming logic in computer science, but it has now expanded to include all sorts of processes. Flowcharts are now widely used for presenting data and helping thinking.

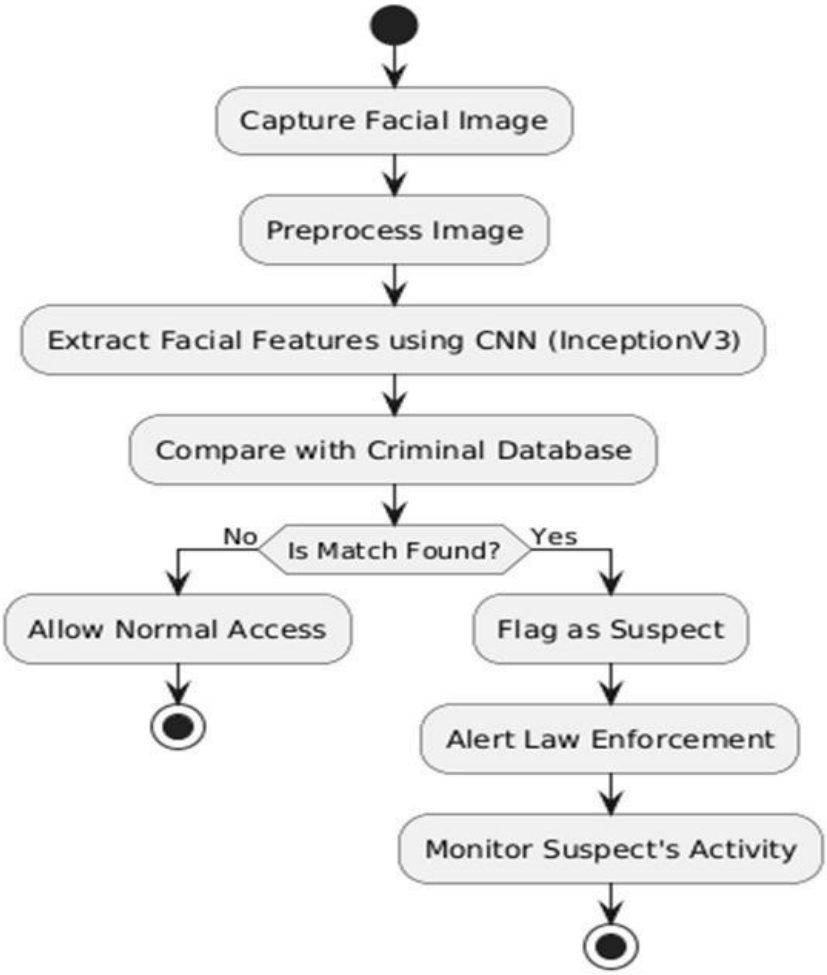


Fig 5.3.4 : Flow Chart Diagram

CHAPTER-6
IMPLEMENTATION

6.1 Technology Description

The different phases involved in image caption generation are as follows:

Step 01 : Preprocessing Of Data

- **Face Detection:** Using Haar Cascade Classifiers to locate faces in images or video frames.
- **Resizing**: All input images are resized to match the required dimensions for InceptionV3, typically 299x299 pixels.
- **Normalization**: The pixel values are normalized (scaled) to the range [0, 1] to ensure stable training.

Step 02: Data Augmentation

To increase the diversity of the dataset and reduce the risk of overfitting, data augmentation techniques are applied, such as:

- **Rotation:** Random rotation of images to simulate different angles.
- **Scaling and Cropping:** Randomly scaling and cropping the images to simulate variations in face size and position.
- **Flipping:** Horizontally flipping images to account for symmetrical features.

Step 03: Feature Extraction

- **Face Embedding Generation:** Once faces are detected and preprocessed, the InceptionV3 model generates embeddings (a vector representation of the face). These embeddings are stored and used for comparison in the criminal database.

Step 04: Evaluation

We use precision as a metric where it helps in understanding the reliability of the system when it alerts about a potential criminal. Higher precision indicates fewer false alarms, which is critical in law enforcement settings.

6.2 Sample Code

```
import Flask, render_template, request, redirect, url_for, flash

import datetime, timedelta

import cv2

import numpy as np

import os

from pathlib import Path

import openpyxl

import base64

from werkzeug.utils import secure_filename

from tensorflow.keras.applications import InceptionV3

from tensorflow.keras.layers import Dense, GlobalAveragePooling2D,
BatchNormalization, Dropout

from tensorflow.keras.models import Model

from tensorflow.keras.optimizers import RMSprop

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.models import load_model

from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
```

```
# Import your data
```

```
from data.criminals import criminals, get_criminal_by_id, get_wanted_criminals,
get_recent_criminals

app = Flask(_name_)
app.secret_key = "criminal_id_secret_key"  # Required for flash messages

# Define constants for paths

DATASET_DIR = "face_recognition_dataset"
MODEL_DIR = "saved_model"
MODEL_NAME = "criminal_identification.keras"
MODEL_PATH = os.path.join(MODEL_DIR, MODEL_NAME)
CLASS_INDICES_PATH = "class_indices.npy"
EXCEL_FILE = "criminals_database.xlsx"
UPLOAD_FOLDER = "uploads"

# Ensure directories exist

for directory in [DATASET_DIR, MODEL_DIR, UPLOAD_FOLDER, 'static/uploads']:
    os.makedirs(directory, exist_ok=True)

# Create symlink for uploads if needed

if not os.path.exists('static/uploads') and os.path.exists('uploads'):
    try:
        os.symlink('../uploads', 'static/uploads')
    except Exception as e:
        print(f"Could not create symlink: {e}")

# Load Class Indices

class_indices = {}
reverse_mapping = {}
if os.path.exists(CLASS_INDICES_PATH):
    try:
        class_indices = np.load(CLASS_INDICES_PATH, allow_pickle=True).item()
        reverse_mapping = {v: k for k, v in class_indices.items()}
        print("Loaded class indices:", reverse_mapping)
    except Exception as e:
        print(f"Error loading class indices: {e}")

def load_data(dataset_dir=DATASET_DIR, img_height=299, img_width=299,
batch_size=32, validation_split=0.3):
```

Data Augmentation for Training

```
train_datagen = ImageDataGenerator(
        rescale=1.0 / 255,
        rotation_range=20,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.15,
        zoom_range=0.15,
        horizontal_flip=True,
        fill_mode='nearest',
        validation_split=validation_split
    )

    # Only Rescaling for Validation (No Augmentation)
    val_datagen = ImageDataGenerator(
        rescale=1.0 / 255,
        validation_split=validation_split
    )

    # Load Training Data
    train_data = train_datagen.flow_from_directory(
        dataset_dir,
        target_size=(img_height, img_width),
        batch_size=batch_size,
        class_mode="categorical",
        subset="training",
        seed=42
    )

    # Load Validation Data
    val_data = val_datagen.flow_from_directory(
        dataset_dir,
        target_size=(img_height, img_width),
        batch_size=batch_size,
        class_mode="categorical",
        subset="validation",
        seed=42
    )

    return train_data, val_data

def create_model(input_shape=(299, 299, 3), num_classes=2, trainable_layers=50,
learning_rate=0.0001, dropout_rate=0.4):
```

Load Pretrained InceptionV3 Model

```
 base_model = InceptionV3(weights='imagenet', include_top=False,
input_shape=input_shape)

    # Freeze all layers initially
    for layer in base_model.layers:
        layer.trainable = False

    # Unfreeze last 'trainable_layers' for fine-tuning
    for layer in base_model.layers[-trainable_layers:]:
        layer.trainable = True

    # Custom Fully Connected Head
    x = base_model.output
    x = GlobalAveragePooling2D()(x)  # Global Pooling instead of Flatten
    x = BatchNormalization()(x)  # Normalize activations
    x = Dense(512, activation='relu')(x)  # Fully connected layer
    x = Dropout(dropout_rate)(x)  # Prevent overfitting

    # Output Layer
    predictions = Dense(num_classes, activation='softmax')(x)

    # Final Model
    model = Model(inputs=base_model.input, outputs=predictions)

    # Compile Model
    model.compile(optimizer=RMSprop(learning_rate=learning_rate),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    # Display Trainable Layers Summary

    trainable_count = sum(1 for layer in model.layers if layer.trainable)
    print(f"\nTotal trainable layers: {trainable_count}/{len(model.layers)}\n")

    return model

def validate_face(face_img):
    """Validate if the detected region is likely to be a face."""
    try:
        # Convert to grayscale
        if len(face_img.shape) == 3:
            gray = cv2.cvtColor(face_img, cv2.COLOR_BGR2GRAY)
```

```
        else:
            gray = face_img

        # 1. Check minimum size
        if face_img.shape[0] < 20 or face_img.shape[1] < 20:
            return False

        # 2. Check aspect ratio (faces should be roughly square)
        aspect_ratio = face_img.shape[1] / face_img.shape[0]
        if aspect_ratio < 0.5 or aspect_ratio > 2.0:
            return False

        # 3. Use eye detection as validation
        eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')
        eyes = eye_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

        # Require at least one eye to be detected
        if len(eyes) < 1:
            return False

        return True
    except Exception as e:
        print(f"Error in face validation: {e}")
        return False


def detect_and_save_faces(images, save_dir, name):
    """Detect faces in uploaded images and save them with padding."""
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
"haarcascade_frontalface_default.xml")
    count = 0
    saved_faces = []  # Store paths of saved face images

    for file in images:
        try:
            # Read uploaded image
            image_bytes = file.read()
            img = cv2.imdecode(np.frombuffer(image_bytes, np.uint8),
cv2.IMREAD_COLOR)
            if img is None:
                continue

            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            # Enhance contrast for better detection
            gray = cv2.equalizeHist(gray)
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        # Enhance contrast for better detection
        gray = cv2.equalizeHist(gray)

        # Detect faces with stricter parameters
        faces = face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=6,
            minSize=(30, 30),
            flags=cv2.CASCADE_SCALE_IMAGE
        )

        for (x, y, w, h) in faces:
            # Extract the face region first for validation
            face_roi = img[y:y+h, x:x+w]

            # Validate if it's actually a face
            if not validate_face(face_roi):
                continue

            # Calculate padding (40% of face width)
            padding_x = int(w * 0.4)
            padding_y = int(h * 0.4)

            # Calculate padded coordinates
            x1 = max(0, x - padding_x)
            y1 = max(0, y - padding_y)
            x2 = min(img.shape[1], x + w + padding_x)
            y2 = min(img.shape[0], y + h + padding_y)

            # Extract padded face region

            face_img = img[y1:y2, x1:x2]
           # Enhance image quality
            lab = cv2.cvtColor(face_img, cv2.COLOR_BGR2LAB)
            l, a, b = cv2.split(lab)
            clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
            cl = clahe.apply(l)
            enhanced = cv2.merge((cl,a,b))
            face_img = cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)
```

```
            # Resize to standard size while maintaining aspect ratio
            target_size = (299, 299)
            h, w = face_img.shape[:2]
            aspect = w/h

            if aspect > 1:
                new_w = target_size[0]
                new_h = int(new_w/aspect)
            else:
                new_h = target_size[1]
                new_w = int(new_h*aspect)

            face_img = cv2.resize(face_img, (new_w, new_h),
interpolation=cv2.INTER_CUBIC)

            # Add black padding to make it square
            delta_w = target_size[0] - new_w
            delta_h = target_size[1] - new_h
            top, bottom = delta_h//2, delta_h-(delta_h//2)
            left, right = delta_w//2, delta_w-(delta_w//2)

            face_img = cv2.copyMakeBorder(face_img, top, bottom, left, right,
                                          cv2.BORDER_CONSTANT, value=[0,0,0])

            # Save the processed face image
            img_path = os.path.join(save_dir, f"{name}_{count}.jpg")
            cv2.imwrite(img_path, face_img, [cv2.IMWRITE_JPEG_QUALITY, 95])
            saved_faces.append(img_path)  # Store the path
            count += 1

        except Exception as e:
            print(f"Error processing image: {e}")
            continue

    # If less than 30 images, duplicate existing ones with slight modifications
    if count > 0 and count < 30:
        original_count = count
        while count < 30:
            # Get an image to duplicate
            source_path = saved_faces[count % original_count]
            source_img = cv2.imread(source_path)
```

```
  if source_img is None:
            continue

        # Apply slight modifications
        # 1. Slight rotation
        angle = np.random.uniform(-5, 5)
        height, width = source_img.shape[:2]
        center = (width/2, height/2)
        rotation_matrix = cv2.getRotationMatrix2D(center, angle, 1.0)
        modified_img = cv2.warpAffine(source_img, rotation_matrix, (width, height))

        # 2. Slight brightness/contrast adjustment
        brightness = np.random.uniform(0.9, 1.1)
        modified_img = cv2.convertScaleAbs(modified_img, alpha=brightness, beta=0)

        # Save the modified duplicate
        img_path = os.path.join(save_dir, f"{name}_{count}.jpg")
        cv2.imwrite(img_path, modified_img, [cv2.IMWRITE_JPEG_QUALITY, 95])
        count += 1

    print(f"Created {30 - original_count} additional images through augmentation")

  return count

def capture_faces_from_camera(save_dir, name, max_faces=30):
  """Capture faces using a webcam with enhanced face detection and padding."""

  os.makedirs(save_dir, exist_ok=True)
  face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
"haarcascade_frontalface_default.xml")

  if face_cascade.empty():
    print("Error: Couldn't load face cascade classifier")
    return 0

  try:
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
      print("Error: Could not open webcam")
      return 0
    count = 0
    consecutive_no_face_frames = 0
    max_no_face_frames = 30
    print("Starting face capture...")
```

```
while count < max_faces:
        ret, frame = cap.read()
        if not ret:
            print("Error: Couldn't read frame from camera")
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        gray = cv2.equalizeHist(gray)

        faces = face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=5,
            minSize=(30, 30)
        )

        if len(faces) > 0:
            consecutive_no_face_frames = 0

            for (x, y, w, h) in faces:
                # Calculate padding (40% of face width)
                padding_x = int(w * 0.4)
                padding_y = int(h * 0.4)

                # Calculate padded coordinates
                x1 = max(0, x - padding_x)
                y1 = max(0, y - padding_y)
                x2 = min(frame.shape[1], x + w + padding_x)
                y2 = min(frame.shape[0], y + h + padding_y)

                # Extract and process face region
                face_img = frame[y1:y2, x1:x2]

                # Enhance image quality
                lab = cv2.cvtColor(face_img, cv2.COLOR_BGR2LAB)
                l, a, b = cv2.split(lab)
                clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
                cl = clahe.apply(l)
                enhanced = cv2.merge((cl,a,b))
                face_img = cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)

                # Resize to standard size while maintaining aspect ratio
                target_size = (299, 299)
                h, w = face_img.shape[:2]
                aspect = w/h
```

```
                if aspect > 1:
                    new_w = target_size[0]
                    new_h = int(new_w/aspect)
                else:
                    new_h = target_size[1]
                    new_w = int(new_h*aspect)

                face_img = cv2.resize(face_img, (new_w, new_h),
interpolation=cv2.INTER_CUBIC)

                # Add black padding to make it square
                delta_w = target_size[0] - new_w
                delta_h = target_size[1] - new_h
                top, bottom = delta_h//2, delta_h-(delta_h//2)
                left, right = delta_w//2, delta_w-(delta_w//2)

                face_img = cv2.copyMakeBorder(face_img, top, bottom, left, right,
                                              cv2.BORDER_CONSTANT, value=[0,0,0])

                # Save the processed face image
                img_path = os.path.join(save_dir, f"{name}_{count}.jpg")
                cv2.imwrite(img_path, face_img, [cv2.IMWRITE_JPEG_QUALITY, 95])
                count += 1
                print(f"Saved {img_path}")

                # Draw rectangle on display frame
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

                # Add small delay
                import time
                time.sleep(0.1)
            else:
                consecutive_no_face_frames += 1
                if consecutive_no_face_frames >= max_no_face_frames:
                    cv2.putText(frame, "No face detected! Please position your face in the
camera",
                                (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

            # Display progress
            cv2.putText(frame, f"Captured: {count}/{max_faces}", (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

            cv2.imshow("Registering Criminal", frame)
```

```
        cv2.destroyAllWindows()

        if count == 0:
            print("No faces detected! Please try again with clearer images.")
        else:
            print(f"Successfully captured {count} faces.")
        return count

    except Exception as e:
        print(f"An error occurred: {str(e)}")
        if 'cap' in locals() and cap.isOpened():
            cap.release()
        cv2.destroyAllWindows()
        return 0

def save_to_excel(criminal_data):
    """Save criminal details to the Excel database."""
    try:
        excel_path = Path(EXCEL_FILE)
        if excel_path.exists():
            wb = openpyxl.load_workbook(excel_path)
            sheet = wb.active
        else:
            wb = openpyxl.Workbook()
            sheet = wb.active
            sheet.append(['Name', 'Age', 'Place', 'Crime Type', 'Crime Details',
                          'Status', 'Registration Date'])

        sheet.append([
            criminal_data['name'],
            criminal_data['age'],
            criminal_data['place'],
            criminal_data['crime_type'],
            criminal_data['crime_details'],
            criminal_data['status'],
            criminal_data['registration_date']
        ])
        wb.save(excel_path)
        return True
    except Exception as e:
        print(f"Error saving to Excel: {e}")
        flash(f"Error saving to Excel: {e}", "error")
        return False
```

```
  def get_wanted_criminals_from_excel():
   """Get wanted criminals from Excel database"""
   wanted_criminals = []
   try:
      if not os.path.exists(EXCEL_FILE):
         return wanted_criminals

      wb = openpyxl.load_workbook(EXCEL_FILE)
      sheet = wb.active
      headers = [cell.value for cell in sheet[1]]

      for row in sheet.iter_rows(min_row=2, values_only=True):
         criminal_data = dict(zip(headers, row))

         if criminal_data.get('Status') == 'Wanted':
            criminal = {
               'id': str(len(wanted_criminals) + 1),  # Generate simple ID
               'name': criminal_data.get('Name', 'Unknown'),
               'age': criminal_data.get('Age', 'N/A'),
               'lastKnownLocation': criminal_data.get('Place', 'Unknown'),
               'status': criminal_data.get('Status', 'Unknown'),
               'crimes': [criminal_data.get('Crime Type', 'Unknown')],
               'dangerLevel': 'High',  # You can add this to Excel or set based on crime type
               'imageUrl': f"/static/uploads/{criminal_data.get('Name')}/profile.jpg"  #
Assuming first image is profile
            }
            wanted_criminals.append(criminal)

      return wanted_criminals[:3]  # Return top 3 wanted criminals
   except Exception as e:
      print(f"Error reading Excel file: {e}")
      return []

def get_all_criminals_from_excel():
   """Get all criminals from Excel database"""
   all_criminals = []
   try:
      if not os.path.exists(EXCEL_FILE):
         return all_criminals

      wb = openpyxl.load_workbook(EXCEL_FILE)
      sheet = wb.active
      headers = [cell.value for cell in sheet[1]]
```

```
            criminal_data = dict(zip(headers, row))

        # Determine danger level based on crime type
        crime_type = criminal_data.get('Crime Type', '').lower()
        danger_level = 'High' if any(x in crime_type for x in ['murder', 'assault', 'robbery'])
else \
                'Medium' if any(x in crime_type for x in ['theft', 'fraud']) else 'Low'

        criminal = {
            'id': str(len(all_criminals) + 1),  # Generate simple ID
            'name': criminal_data.get('Name', 'Unknown'),
            'age': criminal_data.get('Age', 'N/A'),
            'lastKnownLocation': criminal_data.get('Place', 'Unknown'),
            'status': criminal_data.get('Status', 'Unknown'),
            'crimes': [criminal_data.get('Crime Type', 'Unknown')],
            'crimeDetails': criminal_data.get('Crime Details', ''),
            'dangerLevel': danger_level,
            'registrationDate': criminal_data.get('Registration Date', 'N/A'),
            'imageUrl': f'/static/uploads/{criminal_data.get('Name')}/profile.jpg'
        }
        all_criminals.append(criminal)

    return all_criminals
  except Exception as e:
    print(f'Error reading Excel file: {e}')
    return []

def get_criminal_by_id_from_excel(id):
  """Get specific criminal by ID from Excel"""
  all_criminals = get_all_criminals_from_excel()
  return next((criminal for criminal in all_criminals if criminal['id'] == id), None)

@app.route('/')
def index():
  wanted_criminals = get_wanted_criminals_from_excel()
  return render_template('index.html', wanted_criminals=wanted_criminals)

@app.route('/criminals')
def criminals_list():
  search = request.args.get('search', '')
  status = request.args.get('status', 'all')
  danger = request.args.get('danger', 'all')
  filtered_criminals = get_all_criminals_from_excel()
```

```
    if search:
        filtered_criminals = [c for c in filtered_criminals if search.lower() in c['name'].lower()
or
                              any(search.lower() in crime.lower() for crime in c['crimes']) or
                              search.lower() in c['lastKnownLocation'].lower()]

    if status != 'all':
        filtered_criminals = [c for c in filtered_criminals if c['status'].lower() ==
status.lower()]

    if danger != 'all':
        filtered_criminals = [c for c in filtered_criminals if c['dangerLevel'].lower() ==
danger.lower()]

    return render_template('criminals.html', criminals=filtered_criminals)

def process_image(file, model):
    """Process uploaded image and detect criminals."""
    try:
        # Create a filename with timestamp to avoid duplicates
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = secure_filename(file.filename)
        base, ext = os.path.splitext(filename)
        unique_filename = f"{base}_{timestamp}{ext}"

        image_path = os.path.join(UPLOAD_FOLDER, unique_filename)

        # Read image
        image_bytes = file.read()
        img = cv2.imdecode(np.frombuffer(image_bytes, np.uint8), cv2.IMREAD_COLOR)

        if img is None:
            flash("Could not read the uploaded image. Please try with another file.", "error")
            return {}, None

        # Process image with the model
        marked_image, attendance = recognize_faces(img, model)
        # Save processed image
        cv2.imwrite(image_path, marked_image)
        # Convert marked image to Base64 for rendering
        _, buffer = cv2.imencode('.jpg', marked_image)
        marked_image_b64 = base64.b64encode(buffer).decode('utf-8')
        return attendance, marked_image_b64
```

```
  except Exception as e:
      print(f"Error processing image: {e}")
      flash(f"Error processing image: {e}", "error")
      return {}, None

# Face detection and preprocessing functions
def preprocess_face(face_img, target_size=(299, 299)):
    """Preprocess the detected face for CNN prediction with enhanced preprocessing."""
    try:
        # Convert BGR to RGB
        face_img = cv2.cvtColor(face_img, cv2.COLOR_BGR2RGB)

        # Add padding around face
        padding = 30
        h, w = face_img.shape[:2]
        padded = cv2.copyMakeBorder(face_img, padding, padding, padding, padding,
                                    cv2.BORDER_CONSTANT, value=[0, 0, 0])

        # Resize with better interpolation
        face_img = cv2.resize(padded, target_size, interpolation=cv2.INTER_CUBIC)

        # Normalize pixel values
        face_img = face_img.astype('float32')
        face_img = face_img / 255.0

        # Add batch dimension
        face_img = np.expand_dims(face_img, axis=0)

        return face_img
    except Exception as e:
        print(f"Error in preprocessing: {e}")
        return None

def detect_faces(image):
    """Enhanced face detection using both Haar Cascade and adjustable parameters."""
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
"haarcascade_frontalface_default.xml")

    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Enhance image contrast
    gray = cv2.equalizeHist(gray)
```

```
  # Detect faces with different scale factors
    faces1 = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,
minSize=(30, 30))
    faces2 = face_cascade.detectMultiScale(gray, scaleFactor=1.2, minNeighbors=4,
minSize=(30, 30))

    # Combine detections
    faces = np.vstack((faces1, faces2)) if len(faces1) > 0 and len(faces2) > 0 else \
            faces1 if len(faces1) > 0 else faces2

    # Remove duplicate detections
    if len(faces) > 0:
        faces = np.unique(faces, axis=0)

    return faces, gray

def recognize_faces(image, model, confidence_threshold=0.3):
    """Enhanced face recognition with better confidence handling and multiple detections."""
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
"haarcascade_frontalface_default.xml")
    attendance = {}

    # Get all criminals for ID lookup
    all_criminals = get_all_criminals_from_excel()
    criminal_id_map = {criminal['name']: criminal['id'] for criminal in all_criminals}

    # Make a copy of image for drawing
    marked_image = image.copy()

    # Convert to grayscale and enhance contrast
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.equalizeHist(gray)

    # Detect faces with stricter parameters
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=6,
        minSize=(30, 30),
        flags=cv2.CASCADE_SCALE_IMAGE
    )

    for (x, y, w, h) in faces:
        # Extract the face region first for validation
```

```
    face_roi = image[y:y+h, x:x+w]

# Skip if not a valid face
if not validate_face(face_roi):
    continue

# Add padding to face region
padding_x = int(w * 0.4)
padding_y = int(h * 0.4)

x1 = max(0, x - padding_x)
y1 = max(0, y - padding_y)
x2 = min(image.shape[1], x + w + padding_x)
y2 = min(image.shape[0], y + h + padding_y)

face_roi = image[y1:y2, x1:x2]
face_img = preprocess_face(face_roi)

if face_img is None:
    continue

try:
    # Get predictions
    predictions = model.predict(face_img)
    predicted_class = np.argmax(predictions)
    confidence = float(predictions[0][predicted_class])

    # Get top 3 predictions
    top_3_idx = np.argsort(predictions[0])[-3:][::-1]
    top_3_conf = predictions[0][top_3_idx]

    if confidence >= confidence_threshold:
        class_name = reverse_mapping.get(predicted_class, "Unknown")

        if class_name in criminal_id_map:
            criminal_id = criminal_id_map[class_name]

            # Store top 3 matches in attendance
            top_3_matches = []
            for idx, conf in zip(top_3_idx, top_3_conf):
                        match_name = reverse_mapping.get(idx, "Unknown")
```

```
                    if conf >= 0.1:  # Only include matches with >10% confidence
                        top_3_matches.append(f"{match_name} ({conf:.1%})")

                attendance[criminal_id] = {
                    'name': class_name,
                    'confidence': f"Primary Match: {confidence:.1%}",
                    'other_matches': top_3_matches
                }

                # Use strong green color for all detected faces
                color = (0, 255, 0)  # BGR format: Strong Green
            else:
                # Use green for unknown persons too
                color = (0, 255, 0)
            else:
                class_name = "Unknown"
                # Use green even for low confidence
                color = (0, 255, 0)

            # Draw face box and label with consistent green color
            cv2.rectangle(marked_image, (x1, y1), (x2, y2), color, 2)
            label = f"{class_name} ({confidence:.1%})"
            # Draw label background for better visibility
            label_size = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)[0]
            cv2.rectangle(marked_image, (x1, y1-20), (x1 + label_size[0], y1), color, -1)
            # Draw white text on the green background
            cv2.putText(marked_image, label, (x1, y1-5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)

        except Exception as e:
            print(f"Error during prediction: {e}")
            continue

    return marked_image, attendance

def process_video(file, model):
    """Process uploaded video and detect criminals."""
    try:
        # Create unique filename
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = secure_filename(file.filename)
        base, ext = os.path.splitext(filename)
        unique_filename = f"{base}_{timestamp}{ext}"
```

```
    processed_filename = f"processed_{base}_{timestamp}{ext}"

input_path = os.path.join(UPLOAD_FOLDER, unique_filename)
output_path = os.path.join(UPLOAD_FOLDER, processed_filename)

# Save the uploaded file
file.save(input_path)

# Open video
cap = cv2.VideoCapture(input_path)
if not cap.isOpened():
    flash("Could not open the video file", "error")
    return {}, None

# Get video properties
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Create video writer
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(output_path, fourcc, fps, (width, height))

attendance = {}
processed_frames = 0

# Process frames
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Process every 5th frame to speed up processing
    if processed_frames % 5 == 0:
        processed_frame, frame_attendance = recognize_faces(frame, model)

        # Update overall attendance
        for key, value in frame_attendance.items():
            attendance[key] = value
        out.write(processed_frame)
    else:
        out.write(frame)
```

```
                    processed_frames += 1

            # Show progress for long videos
            if frame_count > 0 and processed_frames % 30 == 0:
                progress = (processed_frames / frame_count) * 100
                print(f"Processing video: {progress:.1f}% complete")

        cap.release()
        out.release()

        # Return relative path for template
        video_url = f"/static/uploads/{processed_filename}"
        return attendance, video_url

    except Exception as e:
        print(f"Error processing video: {e}")
        flash(f"Error processing video: {e}", "error")
        return {}, None

def get_training_stats():
    stats = {
        'total_criminals': 0,
        'total_images': 0,
        'train_images': 0,
        'test_images': 0,
        'criminals_data': [],
        'last_trained': None
    }

    try:
        # Load the criminals database
        excel_path = Path(EXCEL_FILE)
        if not excel_path.exists():
            return stats

        wb = openpyxl.load_workbook(excel_path, data_only=True)
        sheet = wb.active
        headers = [cell.value for cell in sheet[1]] if sheet.max_row > 0 else []

        dataset_path = Path(DATASET_DIR)
        if dataset_path.exists():
            for criminal_dir in dataset_path.iterdir():
                if criminal_dir.is_dir():
```

```
                criminal_name = criminal_dir.name

            # Get criminal details from Excel
            criminal_data = {'name': criminal_name, 'registration_date': 'N/A', 'status':
'N/A'}

            for row in sheet.iter_rows(min_row=2, values_only=True):
                row_data = dict(zip(headers, row))
                if row_data.get('Name') == criminal_name:
                    criminal_data['registration_date'] = row_data.get('Registration Date',
'N/A')
                    criminal_data['status'] = row_data.get('Status', 'N/A')
                    break

            # Count images
            image_count = len(list(criminal_dir.glob("*.jpg"))) + \
                      len(list(criminal_dir.glob("*.jpeg"))) + \
                      len(list(criminal_dir.glob("*.png")))

            criminal_data['image_count'] = image_count
            stats['criminals_data'].append(criminal_data)
            stats['total_images'] += image_count

        stats['total_criminals'] = len(stats['criminals_data'])
        stats['train_images'] = int(stats['total_images'] * 0.7)
        stats['test_images'] = stats['total_images'] - stats['train_images']

    # Get last trained date
    model_path = Path(MODEL_PATH)
    if model_path.exists():
        stats['last_trained'] = datetime.fromtimestamp(
            model_path.stat().st_mtime
        ).strftime("%Y-%m-%d %H:%M:%S")

  except Exception as e:
    print(f"Error getting stats: {e}")
    flash(f"Error loading statistics: {e}", "error")

  return stats

@app.route('/criminals/<id>')
def criminal_detail(id):
  criminal = get_criminal_by_id_from_excel(id)
```

```
    if not criminal:
                return render_template('not_found.html'), 404
    return render_template('criminal_detail.html', criminal=criminal)

@app.route('/wanted')
def wanted():
    """Get all wanted criminals from Excel database"""
    wanted_criminals = []
    try:
        if not os.path.exists(EXCEL_FILE):
            return render_template('wanted.html', wanted_criminals=[])

        wb = openpyxl.load_workbook(EXCEL_FILE)
        sheet = wb.active
        headers = [cell.value for cell in sheet[1]]

        for row in sheet.iter_rows(min_row=2, values_only=True):
            criminal_data = dict(zip(headers, row))

            if criminal_data.get('Status') == 'Wanted':
                # Determine danger level based on crime type
                crime_type = criminal_data.get('Crime Type', '').lower()
                danger_level = 'High' if any(x in crime_type for x in ['murder', 'assault',
'robbery']) else \
                               'Medium' if any(x in crime_type for x in ['theft', 'fraud']) else 'Low'

                criminal = {
                    'id': str(len(wanted_criminals) + 1),
                    'name': criminal_data.get('Name', 'Unknown'),
                    'age': criminal_data.get('Age', 'N/A'),
                    'lastKnownLocation': criminal_data.get('Place', 'Unknown'),
                    'status': criminal_data.get('Status', 'Unknown'),
                    'crimes': [criminal_data.get('Crime Type', 'Unknown')],
                    'crimeDetails': criminal_data.get('Crime Details', ''),
                    'dangerLevel': danger_level,
                    'registrationDate': criminal_data.get('Registration Date', 'N/A'),
                    'imageUrl': f"/static/uploads/{criminal_data.get('Name')}/profile.jpg"
                }
                wanted_criminals.append(criminal)

        return render_template('wanted.html', wanted_criminals=wanted_criminals)
    except Exception as e:
        print(f"Error reading Excel file: {e}")
        return render_template('wanted.html', wanted_criminals=[])
```

```
@app.route('/recent')
def recent():
            """Get recently added criminals from Excel database (last 30 days)"""
    recent_criminals = []
    try:
        if not os.path.exists(EXCEL_FILE):
            return render_template('recent.html', recent_criminals=[])
        wb = openpyxl.load_workbook(EXCEL_FILE)
        sheet = wb.active
        headers = [cell.value for cell in sheet[1]]

        # Calculate the date 30 days ago
        thirty_days_ago = datetime.now() - timedelta(days=30)

        for row in sheet.iter_rows(min_row=2, values_only=True):
            criminal_data = dict(zip(headers, row))

            # Convert registration date string to datetime object
            registration_date_str = criminal_data.get('Registration Date')
            try:
                registration_date = datetime.strptime(registration_date_str, "%Y-%m-
%d %H:%M:%S")
            except (ValueError, TypeError):
                continue  # Skip if date is invalid
            # Check if criminal was registered within last 30 days
            if registration_date >= thirty_days_ago:
                # Determine danger level based on crime type
                crime_type = criminal_data.get('Crime Type', '').lower()
                danger_level = 'High' if any(x in crime_type for x in ['murder', 'assault',
'robbery']) else \
                               'Medium' if any(x in crime_type for x in ['theft', 'fraud']) else 'Low'
                criminal = {
                    'id': str(len(recent_criminals) + 1),
                    'name': criminal_data.get('Name', 'Unknown'),
                    'age': criminal_data.get('Age', 'N/A'),
                    'lastKnownLocation': criminal_data.get('Place', 'Unknown'),
                    'status': criminal_data.get('Status', 'Unknown'),
                    'crimes': [criminal_data.get('Crime Type', 'Unknown')],
                    'crimeDetails': criminal_data.get('Crime Details', ''),
                    'dangerLevel': danger_level,
                    'registrationDate': registration_date_str,
                    'imageUrl': f"/static/uploads/{criminal_data.get('Name')}/profile.jpg"
                }
                recent_criminals.append(criminal)
```

```
        # Sort by registration date (newest first)
        recent_criminals.sort(key=lambda x: datetime.strptime(x['registrationDate'], "%Y-
%m-%d %H:%M:%S"),
                              reverse=True)

        return render_template('recent.html', recent_criminals=recent_criminals)
    except Exception as e:
        print(f"Error reading Excel file: {e}")
        return render_template('recent.html', recent_criminals=[])

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/contact')
def contact():
    return render_template('contact.html')

@app.route('/privacy')
def privacy():
    return render_template('privacy.html')

@app.route('/terms')
def terms():
    return render_template('terms.html')

def save_profile_image(face_img, criminal_name):
    """Save the first detected face as profile image"""
    try:
        profile_dir = os.path.join('static/uploads', criminal_name)
        os.makedirs(profile_dir, exist_ok=True)

        profile_path = os.path.join(profile_dir, 'profile.jpg')
        cv2.imwrite(profile_path, face_img)
        return True
    except Exception as e:
        print(f"Error saving profile image: {e}")
        return False

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
```

```
        criminal_data = {
            'name': request.form.get("username"),
            'age': request.form.get("age"),
            'place': request.form.get("place"),
            'crime_type': request.form.get("crime_type"),
            'crime_details': request.form.get("crime_details"),
            'status': request.form.get("status"),
            'registration_date': datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        }

        if not criminal_data['name']:
            flash("Please provide a criminal name", "error")
            return redirect(url_for("register"))

        # Create directories for storing images
        save_dir = os.path.join(DATASET_DIR, criminal_data['name'])
        os.makedirs(save_dir, exist_ok=True)

        registration_method = request.form.get("registration_method", "upload")
        face_count = 0
        profile_saved = False

        if registration_method == "upload":
            uploaded_files = request.files.getlist("images[]")
            if not uploaded_files or uploaded_files[0].filename == '':
                flash("No images uploaded!", "error")
                return redirect(url_for("register"))

            # Process first image to get profile picture
            first_file = uploaded_files[0]
            image_bytes = first_file.read()
            img = cv2.imdecode(np.frombuffer(image_bytes, np.uint8),
cv2.IMREAD_COLOR)

            if img is not None:
                face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
"haarcascade_frontalface_default.xml")
                gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
                faces = face_cascade.detectMultiScale(gray, 1.3, 5, minSize=(30, 30))

                if len(faces) > 0:
                    x, y, w, h = faces[0]  # Get first face
                    face_img = img[y:y+h, x:x+w]
```

```
                profile_saved = save_profile_image(face_img, criminal_data['name'])

        # Reset file pointer for first file
        first_file.seek(0)
        face_count = detect_and_save_faces(uploaded_files, save_dir,
criminal_data['name'])

    else:  # Camera method
        face_count = capture_faces_from_camera(save_dir, criminal_data['name'])
        # Use the first captured image as profile
        captured_images = sorted(os.listdir(save_dir))
        if captured_images:
            first_image = cv2.imread(os.path.join(save_dir, captured_images[0]))
            if first_image is not None:
                profile_saved = save_profile_image(first_image, criminal_data['name'])

    if face_count == 0:
        flash("No faces detected! Please try again with clearer images.", "error")
        if os.path.exists(save_dir):
            try:
                os.rmdir(save_dir)
            except:
                pass
        return redirect(url_for("register"))

    if save_to_excel(criminal_data):
        success_msg = f"Criminal {criminal_data['name']} registered successfully with
{face_count} face images!"
        if not profile_saved:
            success_msg += " (Warning: Could not save profile image)"
        flash(success_msg, "success")
    else:
        flash(f"Faces registered but failed to save data to database.", "warning")

    return redirect(url_for("index"))

  return render_template("register.html")

@app.route('/identify', methods=['GET', 'POST'])
def identify():
  attendance = {}
  marked_image_b64 = None
  video_url = None
```

```
    if request.method == "POST":
        if not os.path.exists(MODEL_PATH):
            flash("Model not found! Train the model first.", "error")
            return redirect(url_for("train"))

        try:
            model = load_model(MODEL_PATH)
        except Exception as e:
            flash(f"Failed to load model: {e}", "error")
            return redirect(url_for("index"))

        input_type = request.form.get("input_type", "image")

        if input_type == "image":
            file = request.files.get("image")
            if not file or file.filename == '':
                flash("No image uploaded!", "error")
                return redirect(url_for("identify"))
            attendance, marked_image_b64 = process_image(file, model)
        else:  # video
            file = request.files.get("video")
            if not file or file.filename == '':
                flash("No video uploaded!", "error")
                return redirect(url_for("identify"))
            attendance, video_url = process_video(file, model)

        if attendance:
            flash("Criminals identified successfully! Click on their names for full details.",
"success")
        else:
            flash("No criminals identified in the input.", "info")

    return render_template("identify.html",
                           attendance=attendance,
                           marked_image=marked_image_b64,
                           video_url=video_url)

@app.route('/train', methods=['GET', 'POST'])
def train():
    stats = get_training_stats()
    if request.method == "POST":
        try:
```

```
    os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)
train_data, val_data = load_data(DATASET_DIR)

if not train_data.class_indices:
    flash("No classes found in dataset. Please register criminals first.", "error")
    return redirect(url_for("index"))

np.save(CLASS_INDICES_PATH, train_data.class_indices)
global class_indices, reverse_mapping
class_indices = train_data.class_indices
reverse_mapping = {v: k for k, v in class_indices.items()}

num_classes = len(train_data.class_indices)
print(f"Training model with {num_classes} classes")

model = create_model(input_shape=(299, 299, 3), num_classes=num_classes)

callbacks = [
    ModelCheckpoint(
        filepath=MODEL_PATH,
        save_best_only=True,
        monitor="val_accuracy",
        mode="max"
    ),
    EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True
    )
]

history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=20,
    callbacks=callbacks,
    verbose=1
)

model.save(MODEL_PATH)
flash("Model trained successfully!", "success")
```

```
except Expection e:
            flash(f"Training failed: {str(e)}", "danger")
            print(f"Training error: {e}")

        return redirect(url_for("index"))

    return render_template("train.html", stats=stats)
```

Sample data module (data/criminals.py)

```
if _name_ == '_main_':
    app.run(debug=True)
```

CHAPTER-7
SCREEN SHOTS

Fig7.1 : Home Page

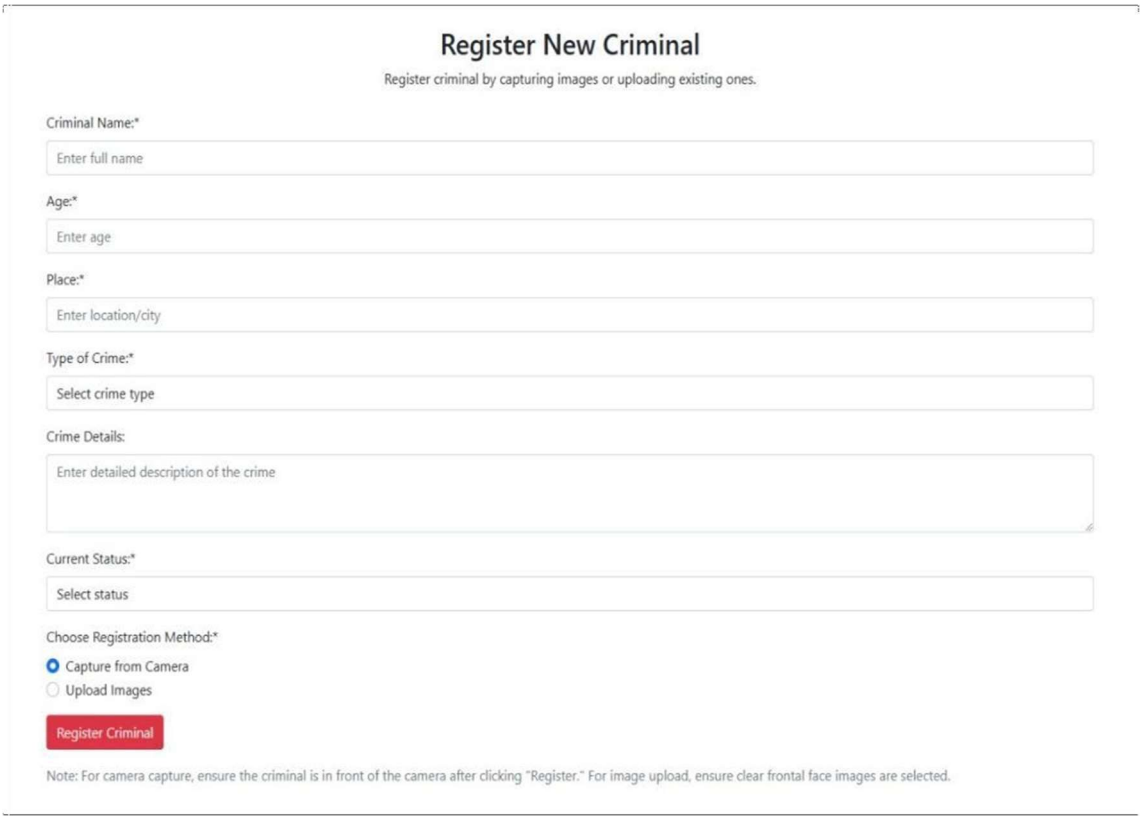


Fig7.2 : Criminal Registration Page

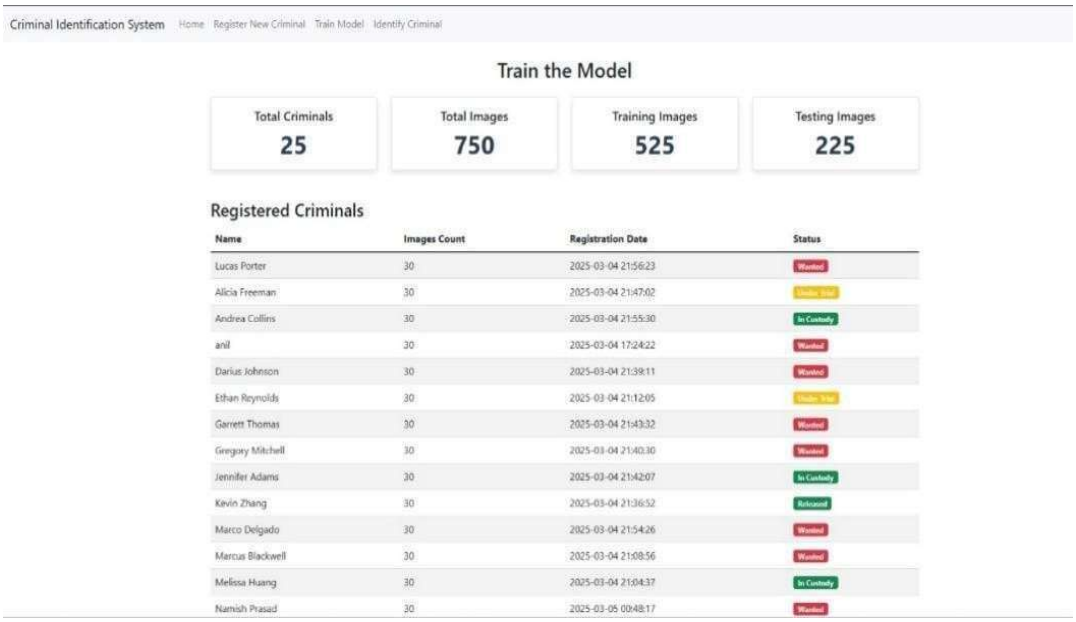


Fig7.1 : Model Training Page

```
Epoch 1/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 21s 3s/step - accuracy: 0.3907 - loss: 1.8277 - val_accuracy: 0.5833 - val_loss: 1.3629
Epoch 2/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 11s 2s/step - accuracy: 0.8617 - loss: 0.4550 - val_accuracy: 0.7500 - val_loss: 1.1541
Epoch 3/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 12s 2s/step - accuracy: 0.9920 - loss: 0.1363 - val_accuracy: 0.7667 - val_loss: 1.0266
Epoch 4/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 12s 2s/step - accuracy: 0.9735 - loss: 0.1299 - val_accuracy: 0.7833 - val_loss: 0.9248
Epoch 5/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 12s 2s/step - accuracy: 0.9773 - loss: 0.0944 - val_accuracy: 0.8333 - val_loss: 0.8193
Epoch 6/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 11s 2s/step - accuracy: 0.9943 - loss: 0.0486 - val_accuracy: 0.8833 - val_loss: 0.7457
Epoch 7/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 11s 3s/step - accuracy: 1.0000 - loss: 0.0699 - val_accuracy: 0.8833 - val_loss: 0.6973
Epoch 8/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 12s 3s/step - accuracy: 0.9922 - loss: 0.0397 - val_accuracy: 0.9167 - val_loss: 0.6252
Epoch 9/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 12s 2s/step - accuracy: 1.0000 - loss: 0.0251 - val_accuracy: 0.9667 - val_loss: 0.5539
Epoch 10/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 11s 3s/step - accuracy: 0.9901 - loss: 0.0899 - val_accuracy: 0.9833 - val_loss: 0.4920
Epoch 11/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 10s 2s/step - accuracy: 1.0000 - loss: 0.0154 - val_accuracy: 0.9667 - val_loss: 0.4473
Epoch 12/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 11s 2s/step - accuracy: 0.9712 - loss: 0.0559 - val_accuracy: 1.0000 - val_loss: 0.4005
Epoch 13/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 10s 2s/step - accuracy: 1.0000 - loss: 0.0166 - val_accuracy: 1.0000 - val_loss: 0.3606
Epoch 14/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 11s 2s/step - accuracy: 0.9762 - loss: 0.0298 - val_accuracy: 1.0000 - val_loss: 0.3136
Epoch 15/20
```

Fig7.2 : Training Of Model

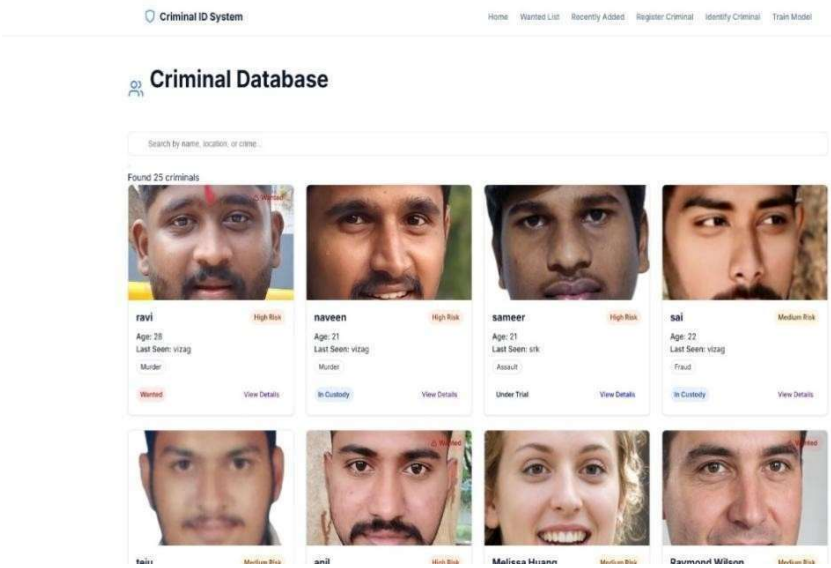


Fig7.3 : Criminal Database

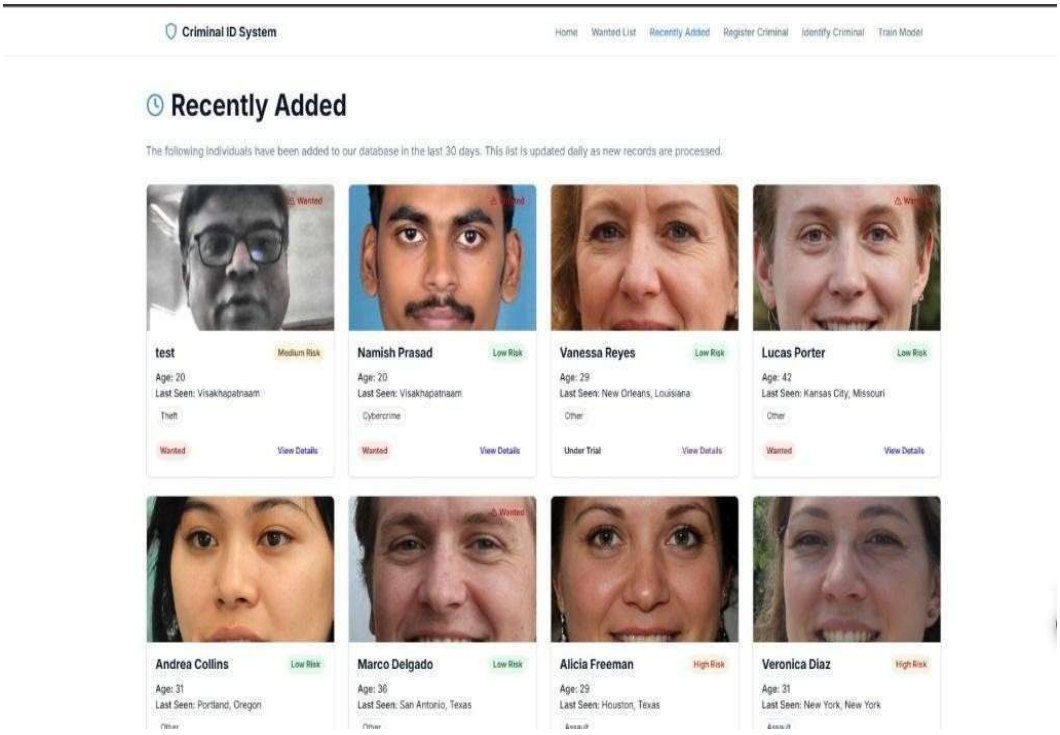


Fig7.4 : Recently Added Criminals

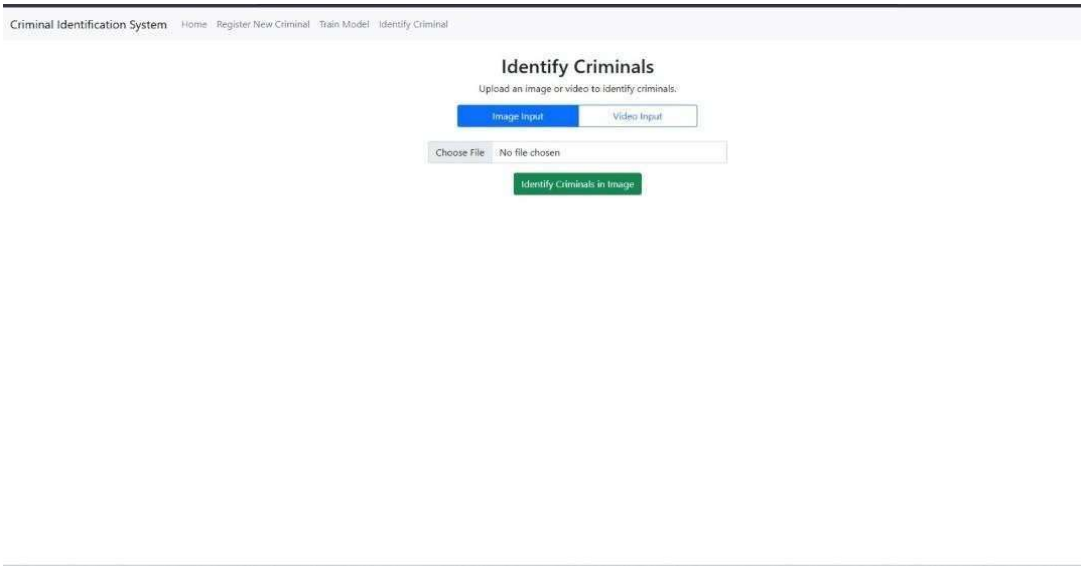


Fig7.5 : Identification Of Criminals

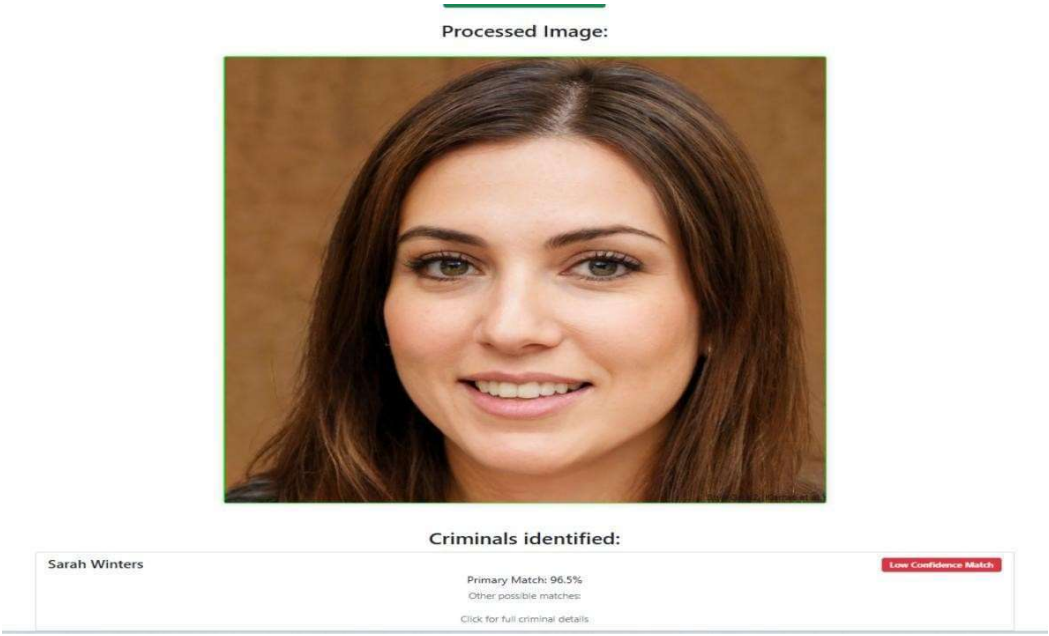


Fig7.6: Criminal Identified

.

CHAPTER-8
SYSTEM TESTING

8.1 Introduction

Testing is a critical phase in the development of the Criminal Identification System to ensure that the facial recognition model performs accurately and reliably in real-world scenarios. This chapter outlines the comprehensive testing methodology, evaluation metrics, and results obtained during the testing phase of the system.

8.2 Testing Methodology

The testing of the Criminal Identification System was conducted through a multi-phase approach

8.2.1 Unit Testing

Individual components of the system were tested independently to verify their functionality:

- Face Detection Module: Tested with various image inputs to ensure accurate detection of faces with different poses, lighting conditions, and occlusions.
- Image Preprocessing Functions: Validated for correct enhancement, normalization, and standardization of input images.
- Model Prediction Pipeline: Verified for proper loading of the trained model and generation of predictions.
- Database Integration: Tested for accurate storage and retrieval of criminal records.

8.2.2 Integration Testing

The integration of different system components was tested to ensure seamless interaction:

- Input Processing to Face Detection: Verified that uploaded images and video streams were correctly processed for face detection.
- Face Detection to Feature Extraction: Tested the handoff of detected and preprocessed face images to the InceptionV3 model.
- Feature Extraction to Classification: Validated that the extracted features were

properly classified by the model.

- Classification to Output Generation: Ensured that the classification results were correctly translated into user-friendly outputs.

8.2.3 System Testing

The complete system was tested end-to-end to verify overall functionality:

- Criminal Registration: Tested the process of registering new criminals in the system, including image upload/capture and data storage.
- Criminal Identification: Validated the system's ability to identify criminals from new images and video inputs.
- Web Interface: Tested all user interface elements for proper functionality and responsiveness.

8.2.4 Performance Testing

The system was subjected to performance testing to ensure it meets the required standards:

- Response Time: Measured the time taken to process inputs and generate outputs.
- Throughput: Tested the system's capacity to handle multiple requests simultaneously.
- Resource Utilization: Monitored CPU, memory, and disk usage during system operation.

8.3 Error Analysis

Analysis of the errors made by the model revealed several patterns:

1. Similar Facial Features: The model occasionally confused criminals with similar facial features, particularly those who were related (siblings or parents and children).
2. Poor Lighting Conditions: Images captured in extreme lighting conditions (too bright or too dark) resulted in lower identification accuracy.
3. Heavy Facial Occlusions: Faces with significant occlusions such as large sunglasses, face masks, or heavy beards led to decreased recognition performance.

4. Extreme Pose Variations: Face images with extreme pose variations (looking up, down, or sideways) resulted in lower accuracy compared to frontal faces.

8.4 Testing Challenges and Solutions

8.4.1 Challenges

Several challenges were encountered during the testing phase:

- Limited Test Data: For some criminals, only a limited number of test images were available.
- Quality Variations: The quality of images varied significantly across the dataset.
- Processing Time: Initial implementations had longer processing times for video inputs.
- False Positives: The system occasionally identified non-criminals as criminals in the database.

8.4.2 Solutions Implemented

The following solutions were implemented to address the testing challenges:

- Data Augmentation: Additional test data was generated through augmentation techniques such as rotation, brightness adjustments, and slight transformations.
- Enhanced Preprocessing: The image preprocessing pipeline was improved to handle quality variations better.
- Optimization: Code optimizations were implemented to reduce processing time, including selective frame processing for videos.
- Confidence Threshold Adjustment: A higher confidence threshold was implemented to reduce false positives at the cost of slightly lower recall.

CHAPTER-9
CONCLUSION & FUTURE ENHANCEMENTS

CONCLUSION:

- The Automated Criminal Detection System using the InceptionV3 model demonstrates the potential of deep learning based facial recognition systems in improving law enforcement and public safety.
- The system's high accuracy, precision, and recall, coupled with its real-time processing capabilities, make it a valuable tool for criminal identification in dynamic environments.
- By integrating criminal history data and facial recognition, the system not only provides an efficient method for detecting known criminals but also contributes to more reliable and transparent law enforcement practices.In comparison with existing models and prior studies, the InceptionV3 model offers superior performance in terms of both recognition accuracy and real-time processing.
- This suggests that InceptionV3 is a highly suitable architecture for criminal detection tasks, balancing speed, accuracy, and reliability.

FUTURE ENHANCEMENTS:

1. **Dataset Expansion and Diversity**:

   To improve the model's generalization ability, future work should focus on expanding the training dataset to include a **more diverse range of facial images**. This includes covering various demographics, lighting conditions, camera angles, and facial expressions. A **larger, more representative dataset** will help the system achieve **better performance** across diverse real-world scenarios.

2. **Improving Robustness to Environmental Factors**:

   Enhancing the model's ability to handle **poor lighting**, **extreme angles**, and **occluded faces** would make the system more reliable in dynamic, real-world environments. Future work could incorporate **data augmentation techniques** such as **random cropping**, **flipping**, **adjusting brightness**, and **adding noise** to simulate real-world conditions, helping the model become more robust to such challenges.

3. **Addressing False Negatives**:

Future improvements could involve integrating **multi-modal recognition** techniques, such as combining **facial recognition with voice recognition** or **gait analysis**. This would provide additional verification layers to reduce the likelihood of **false negatives** and increase the accuracy of the identification system.

4. **Privacy Protection and Ethical Considerations**:

Given the growing concerns about **privacy**, future iterations of the system should implement **strong data protection measures** to comply with **privacy laws** (e.g., GDPR, CCPA). Additionally, ethical guidelines should be established to ensure that the technology is used responsibly, ensuring that individuals' rights to privacy are respected while using the system for public safety purposes. This could include **opt-in policies**, **transparent usage protocols**, and **regular audits** of the system's deployment.

5. **Optimizing for Computational Efficiency**:

While **InceptionV3** provides high performance, it is computationally demanding. Future work could focus on optimizing the model for **resource-constrained devices** by using **lighter models** (such as **MobileNet** or **EfficientNet**) that offer similar accuracy but with lower computational requirements. Alternatively, **model pruning** or **quantization** techniques can be employed to reduce the model's size and speed up inference time without sacrificing much in terms of accuracy.

6. **Scalability and Real-World Testing**:

Finally, further testing and refinement of the system in **real-world environments** will be crucial to assess its **scalability** in large-scale surveillance applications. This includes testing in various settings like **airports**, **stadiums**, and **city streets** to evaluate performance in crowded, uncontrolled environments. Continuous monitoring and **fine-tuning** of the system will be necessary for its successful deployment in public spaces.

CHAPTER-10
REFERENCES

REFERENCES:

[1] O. M. Parkhi, A. Vedaldi, and A. Zisserman, “Deep face recognition,” Procedings of the British Machine Vision Conference 2015, 2015.

[2] Yan Ke and R. Sukthankar, “PCA-SIFT: A more distinctive representation for local image descriptors,” Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.

[3] R. Chellappa, C. L. Wilson, and S. Sirohey, “Human and machine recognition of faces: A survey,” Proceedings of the IEEE, vol. 83, no. 5, pp. 705–741, 1995.

[4] Xiaofei He, Shuicheng Yan, Yuxiao Hu, P. Niyogi, and Hong-Jiang Zhang, “Face recognition using Laplacianfaces,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 27, no. 3, pp. 328 340, 2005.

[5] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015.

[6] Y. Wen, K. Zhang, Z. Li, and Y. Qiao, “A discriminative feature learning approach for deep face recognition,” Computer Vision – ECCV 2016, pp. 499 515, 2016.

[7] Y. Sun, D. Liang, X. Wang, and X. Tang, “DeepID3: Face Recognition with Very Deep Neural Networks.”

[8] V. Bruce, Z. Henderson, K. Greenwood, P. J. Hancock, A. M. Burton, and P. Miller, “Verification of face identities from images captured on video.,” Journal of Experimental Psychology: Applied, vol. 5, no. 4, pp. 339–360, 1999.

[9] Z. Henderson, V. Bruce, and A. M. Burton, “Matching the faces of robbers captured on video,” Applied Cognitive Psychology, vol. 15, no. 4, pp. 445–464, 2001.

[10] V. Bruce, Z. Henderson, C. Newman, and A. M. Burton, “Matching identities of familiar and unfamiliar faces caught on CCTV images.,” Journal of Experimental Psychology: Applied, vol. 7, no. 3, pp. 207 218, 2001.

[11] A. M. Megreya and A. M. Burton, “Unfamiliar faces are not faces: Evidence from a matching task,” Memory & Cognition, vol. 34, no. 4, pp. 865 876, 2006.

[12] A. M. Burton, S. Wilson, M. Cowan, and V. Bruce, “Face recognition in poor-quality video: Evidence from security surveillance,” Psychological Science, vol. 10, no. 3, pp. 243–248, 1999.